



Graph-based reinforcement learning for software-defined networking traffic engineering

Jingwen Lu¹ · Chaowei Tang¹ · Wenyu Ma¹ · Wenjuan Xing^{1,2}

Received: 10 April 2025 / Accepted: 21 June 2025 / Published online: 21 July 2025
© The Author(s) 2025

Abstract

With the continuous expansion of global Internet infrastructure, wide area networks play a crucial role in transmitting traffic between multiple data centers and users worldwide. However, efficient traffic management has become a core challenge due to the high costs of building and maintaining these networks. Traditional traffic engineering methods based on linear programming achieve optimal solutions but suffer from exponential computational complexity growth with network size, making them impractical for real-time applications in large-scale networks. Recent machine learning approaches show promise but still face fundamental limitations in handling complex network constraints and maintaining performance across different network scales. This paper proposes GRL-TE (Graph-based Reinforcement Learning for Traffic Engineering), a novel framework that achieves near-optimal performance while maintaining computational efficiency across diverse network scales. GRL-TE introduces three key innovations: (1) TopoFlowNet, a graph neural network architecture that models WANs as bipartite graphs with edge nodes representing physical links and path nodes representing candidate paths, enabling efficient bidirectional information propagation through GINConv layers while MLP modules handle collaborative relationships among paths serving the same demand; (2) A one-step A2C mechanism specifically designed for TE with immediate reward structure, eliminating the need for future state estimation and significantly simplifying training; (3) Integration of ADMM as a post-processing step to iteratively reduce constraint violations while improving solution quality. Extensive experiments on six real-world WAN topologies ranging from 12 to 1,739 nodes demonstrate that GRL-TE achieves an overall average demand satisfaction rate of 89.36%, outperforming state-of-the-art learning-based methods (Teal: 82.04%, Figret: 82.20%) and the clustering-based NCFLOW (76.48%), while providing 3–4 orders of magnitude speedup compared to LP solvers on large-scale networks. The framework maintains robust performance under link failures and meets real-time scheduling requirements for production deployment.

Keywords Traffic engineering · Graph neural networks · Deep reinforcement learning · Software-defined networks

✉ Chaowei Tang
cwtang@cqu.edu.cn
Jingwen Lu
ljw@cqu.edu.cn
Wenyu Ma
mwy@stu.cqu.edu.cn
Wenjuan Xing
xingwj@chinatelecom.cn

¹ School of Microelectronics and Communication Engineering, Chongqing University, No. 55 Daxuecheng South Road, Shapingba District, Chongqing 401331, China

² Beijing Research Institute, China Telecom Corporation Limited, No. 118, Xizhimennei Street, Xicheng District, Beijing 102209, China

1 Introduction

With the continuous expansion of global internet infrastructure, wide area networks (WANs) play a crucial role in transmitting traffic between multiple data centers and users worldwide (Bernárdez et al. 2021; Awduche et al. 2002). These large-scale WANs enable interregional communication, but efficient traffic management has become a core challenge for network service providers due to the high costs associated with building and maintaining these networks (Marouani et al. 2024). Traffic engineering (TE) is a key technology aimed at optimizing network resource allocation. By dynamically adjusting traffic paths, TE not only improves overall network utilization (Hong et al. 2013, 2018) but also

reduces latency (Schlinker et al. 2017; Yap et al. 2017) and enhances network fault tolerance (Abuzaid et al. 2021; Bogle et al. 2019; Liu et al. 2014). Consequently, TE has become an essential part of modern WAN operations.

Software-defined networking (SDN), characterized by the decoupling of the control and data planes, offers network operators a centralized view and dynamic control capabilities. Unlike traditional distributed networks, SDN allows centralized controllers to quickly adapt to changing network conditions, making it particularly suitable for efficient TE tasks (Mehraban and Yadav 2024). By leveraging SDN's programmability, operators can dynamically adjust traffic flows to meet evolving demands and swiftly mitigate network failures. However, current SDN-based TE methods still face significant challenges, particularly in terms of scalability, real-time responsiveness, and optimization accuracy.

Over the past decade, as network scales have expanded and traffic demands have continued to evolve, TE has become increasingly complex. Traditional TE methods rely on centralized optimization solvers based on linear programming (LP) for dynamic resource allocation. While LP can achieve theoretical optimal solutions, its computational complexity grows exponentially with network size (Jain et al. 2013), making it impractical for large-scale real-time applications. For instance, our experiments show that LP requires over 2,500 seconds for the Kdl network (754 nodes) and exceeds 10,000 seconds for the ASN network (1,739 nodes). Beyond these computational complexity limitations, existing acceleration schemes also exhibit significant scale-dependent limitations - for instance, NCFlow (Abuzaid et al. 2021) performs well on small networks (98.9% on B4) but degrades dramatically on large-scale networks (44.78% on ASN) due to its clustering-based decomposition approach. Furthermore, these optimization-based methods struggle to effectively model the complex coupling between network capacity constraints and dynamic traffic patterns, typically optimizing them separately rather than jointly considering their interactions.

Recent machine learning (ML) approaches (Bernárdez et al. 2021; Marouani et al. 2024), particularly deep reinforcement learning (DRL) (Schrittwieser et al. 2020), have shown promise in transforming TE by learning network traffic patterns. However, existing DRL-based methods still face fundamental limitations. Traditional DRL architectures rely on cumulative rewards and future state predictions, which not only complicate training but also fail to align with the nature of TE decisions where traffic allocation at each time step only affects current performance. Moreover, neural networks inherently cannot guarantee strict constraint satisfaction, potentially leading to capacity violations and infeasible solutions in production environments. Additionally, some existing learning-based methods fail to scale to large networks, either showing significant performance

degradation or being unable to complete execution due to computational limitations.

To address these challenges, this paper introduces GRL-TE (Graph-based Reinforcement Learning for Traffic Engineering), a novel framework that achieves near-optimal performance while maintaining computational efficiency across diverse network scales, from small private WANs to large-scale networks with thousands of nodes.

Specifically, GRL-TE makes the following key contributions:

- **TopoFlowNet Architecture:** We propose a novel graph neural network architecture that models WANs as bipartite graphs, with edge nodes representing physical links and path nodes representing candidate paths. This design enables efficient bidirectional information propagation through GINConv layers to capture capacity constraints and resource competition, while MLP modules handle collaborative relationships among paths serving the same demand. This unified representation allows the model to learn both network topology and traffic patterns simultaneously.
- **One-Step A2C Mechanism:** We design a simplified A2C framework specifically tailored for TE characteristics. Unlike traditional A2C that relies on temporal difference learning and discount factors, our one-step mechanism focuses solely on immediate rewards, eliminating the need for future state estimation. This design not only simplifies training but also aligns perfectly with TE scenarios where current routing decisions do not affect future traffic demands.
- **ADMM Integration for Quality Enhancement:** To address the inherent limitation of neural networks in guaranteeing constraint satisfaction, we integrate ADMM as a post-processing refinement step. ADMM iteratively reduces constraint violations while improving the quality of DRL-generated allocations through its optimization process. This hybrid approach makes GRL-TE suitable for production deployment.

The structure of this paper is as follows: Section 2 reviews the research background and related work in the field of TE. Section 3 describes the SDN-based TE implementation method. Section 4 details the design and implementation of the GRL-TE method. Section 5 evaluates GRL-TE's performance across different network topologies and compares it with existing optimization schemes. Section 6 summarizes the main findings and outlines future research directions.

2 Related work

TE aims to optimize network traffic allocation to improve resource utilization, with core objectives including minimiz-

ing maximum link utilization, reducing end-to-end delay, enhancing throughput, and strengthening fault tolerance (Hong et al. 2013, 2018; Yap et al. 2017; Bogle et al. 2019; Liu et al. 2014; Jain et al. 2013; Xu et al. 2023; Krishnaswamy et al. 2022). With the continuous growth of network scale and service demands, TE has become a key research direction connecting academia and industry (Abuzaid et al. 2021; Nance-Hall et al. 2023; Jin et al. 2016; Laoutaris et al. 2011).

2.1 Traditional optimization-based methods

Early research on TE primarily relied on mathematical programming (linear and convex optimization) and algorithmic models (shortest path, minimum cost flow) for traffic control. Fortz et al. (2002) demonstrated that intelligent configuration of static link weights in large IP networks using open shortest path first (OSPF) protocol (Fortz and Thorup 2000) could achieve near-optimal traffic allocation. Zhang et al. (2009) constructed LP based on minimum cost flow models, combining the flexibility of multi-protocol label switching (MPLS) tunnels with the stability of OSPF, achieving fine-grained traffic splitting while suppressing routing oscillations. Although these traditional methods established the foundation for TE, they exhibited notable limitations when dealing with large-scale dynamic network environments. Static optimization methods have limited adaptability to network topology changes (such as link failures); once the network structure changes, the original optimization solution may become completely ineffective and require recalculation. Furthermore, the computational complexity of LP models increases rapidly with network size, making them impractical for real-time applications in large-scale networks—as our experiments have shown, LP takes several hours to process networks with thousands of nodes.

2.2 Dynamic control-based methods

Dynamic control-based methods utilize real-time closed-loop feedback mechanisms to rapidly respond to topology and traffic fluctuations. NCFlow (Abuzaid et al. 2021) achieves fast path reconfiguration through innovative end-to-end flow encoding techniques, significantly enhancing network adaptability. Narayanan et al. (2021) employ partitioned parallel processing with accelerated control loops to dynamically adjust link weights and traffic splitting ratios online, effectively reducing network congestion probability. However, these methods face inherent limitations: they rely on predefined control rules rather than learning from data, require extensive domain expertise for parameter tuning, and most critically, their performance deteriorates significantly as network scale increases.

2.3 Data-driven methods

In recent years, data-driven methods—particularly ML and reinforcement learning (RL) technologies—have become the focus of adaptive TE research (Gui et al. 2024). Bernárdez et al. (2021) proposed a TE system based on distributed multiagent RL (MARL) and GNN, aiming to dynamically adjust OSPF link weights to minimize maximum link utilization. However, this research was primarily validated on small-scale network topologies, with the largest test network—GÉANT2 (Rusek et al. 2019)—containing only 72 links, leaving its scalability in actual backbone networks with thousands of links yet to be proven. Geng et al. (2021) introduced RedTE, a MARL-based framework enabling routers to make distributed TE decisions using only local information, achieving performance close to centralized solutions. However, RedTE's training process is computationally intensive, requiring approximately 12 hours even on high-performance NVIDIA A6000 GPUs for KdI networks, creating a practical deployment barrier in resource-constrained environments.

With SDN (McKeown et al. 2008) decoupling the control plane from the data plane (Marouani et al. 2024), researchers have begun exploring intelligent TE methods in SDN environments (Ding et al. 2024; Swaminathan et al. 2021; Guo et al. 2021). Ma et al. (2025) proposed the FRRL method, which integrates RL with topology difference vector (TDV) encoding to accelerate route recalculation during link failures in hybrid SDN deployments. However, its evaluation benchmarks primarily consist of traditional or early-stage methods such as OSPF, SRTE (Tian et al. 2020), and ROAR (Guo et al. 2021), lacking comparisons with more recent state-of-the-art approaches. Recently, Guo et al. (2024) proposed MATE, a distributed MARL framework for hybrid SDN environments. MATE significantly improves network robustness by dynamically adjusting routing strategies. While it demonstrates excellent performance in failure recovery and load balancing, MATE also highlights key challenges associated with distributed MARL: excessive training resource demands (e.g., 85 hours of offline training on the 34-node Arnes topology), complex inter-agent coordination, and stringent communication quality requirements. These factors collectively hinder its practical deployment in large-scale networks.

In summary, existing TE methods face a fundamental trade-off: traditional optimization methods achieve optimal solutions but become computationally intractable for large networks; dynamic control methods offer real-time performance but lack learning capabilities and scalability; data-driven methods show adaptability but struggle with training efficiency and deployment challenges in large-scale networks.

To address these challenges, this paper proposes GRL-TE, which innovatively combines three complementary techniques: (1) graph neural networks (GNNs) for efficient

network state representation, (2) single-agent RL that significantly reduces complexity compared to MARL approaches, and (3) ADMM integration to iteratively reduce constraint violations and improve solution quality. Through comprehensive evaluations on large-scale networks (up to 1,739 nodes) and multi-link failure scenarios, we demonstrate GRL-TE's superior efficiency, scalability, and robustness for real-world WAN deployment.

3 SDN-based TE implementation method

The SDN-based TE method proposed in this paper is shown in Fig. 1. By decoupling the control plane from the data plane, SDN promotes centralized network management and dynamic resource optimization.

The control plane consists of three components: the monitoring module, the TE controller, and the flow rules translation module. The monitoring module collects global network status information, including topology and traffic demands. The TE controller uses this data to perform optimization calculations and generate traffic allocation strategies. The flow rules translation module then converts these strategies into specific flow table rules, which are sent to data plane devices for execution.

The data plane is composed of devices that handle actual network traffic. These devices forward traffic according to the flow table rules issued by the control plane and provide real-time network status feedback for the subsequent optimization process of the control plane.

The GRL-TE method in this paper mainly focuses on the design and optimization of the TE controller in the control plane, aiming to provide a fast and high-quality traffic allocation strategy.

The SDN-based TE method consists of three main steps:

3.1 Collecting traffic matrix information

The monitoring module in the control plane is responsible for collecting the network's traffic matrix (TM), which

represents the traffic demand between source and destination nodes. This matrix is generated by traffic measurement or prediction platforms deployed throughout the network and uploaded to the control plane. It reflects the network's traffic demands and serves as the foundation for routing optimization by the TE controller.

3.2 Running the GRL-TE optimization method

After receiving the TM, the TE controller runs the GRL-TE optimization method, which integrates three core components: TopoFlowNet, DRL, and ADMM. Firstly, TopoFlowNet converts network topology and traffic requirements into compact graph embedding features. Then, the DRL model quickly generates an initial traffic allocation strategy based on these characteristics. Finally, the ADMM algorithm further optimizes the DRL output to improve the quality of the allocation scheme.

3.3 Distributing routing strategies

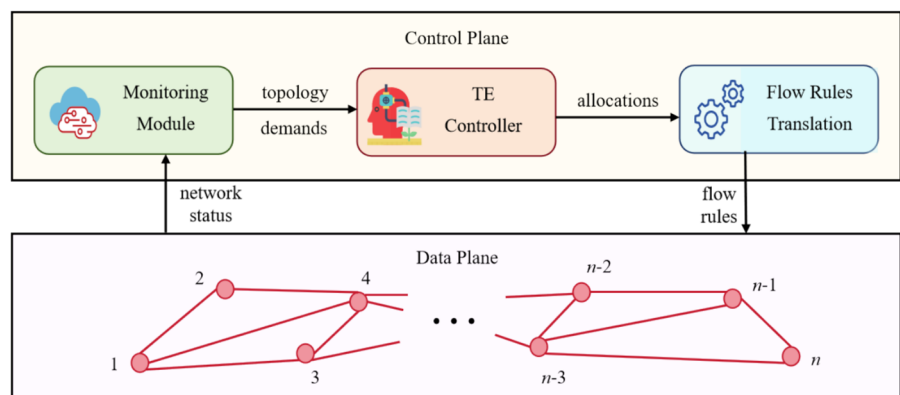
The flow rules translation module converts the traffic allocation strategy generated by the TE controller into specific flow table rules that can be executed by the data plane. These rules are then sent to the data plane devices. After receiving the updated flow table rules, the data plane devices forward traffic according to the new rules.

Thanks to the SDN architecture, routing optimization and strategy distribution within the control plane can be performed concurrently, thus enhancing the system's efficiency and response speed (Ding et al. 2024).

4 Design of the GRL-TE method

In this section, we introduce the design of the GRL-TE method. GRL-TE is a method focused on TE controller design, aiming to provide fast and efficient solutions for TE controllers by integrating TopoFlowNet, DRL, and ADMM.

Fig. 1 SDN-based TE scenario



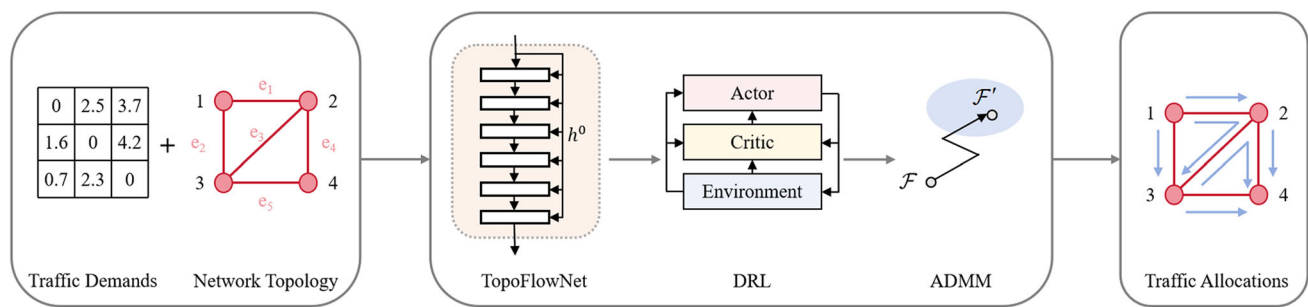


Fig. 2 Work flow of the GRL-TE

Its workflow is shown in Fig. 2. When the TE controller receives new traffic demands, TopoFlowNet converts link capacities and traffic demands into compact feature vectors. These embeddings not only retain the graph structure of the network topology but also encode traffic-related features, providing robust support for subsequent traffic allocation.

These feature vectors are then input into the DRL model that employs the A2C architecture for policy optimization. The Actor generates initial traffic allocations based on the feature embeddings provided by TopoFlowNet. At the same time, the Critic evaluates the quality of the current policy by calculating the value function and guides the Actor to continuously optimize its policy.

However, during the deployment phase, the traffic allocations generated by the DRL model may violate certain link capacity constraints, leading to packet loss or suboptimal TE performance. To address these issues and further improve solution quality, GRL-TE incorporates a classical constrained optimization method—ADMM—after the DRL model. ADMM iteratively refines the traffic allocation to reduce constraint violations while maintaining allocation quality. Despite the significant improvements from ADMM's fixed-iteration optimization, minor constraint violations may still persist due to numerical precision and the finite number of iterations. Therefore, as a final step, the ADMM-refined allocation undergoes a constraint-aware action adjustment process (detailed in Section 4.3.1) that enforces hard constraints through demand and capacity validation, ensuring the strict feasibility of the final solution.

Through TopoFlowNet's efficient feature extraction, DRL's fast decision-making, and ADMM's constraint optimization, GRL-TE fully leverages the computational efficiency and hardware acceleration advantages of deep learning while guaranteeing constraint satisfaction. This design enables TE controllers to rapidly generate high-quality, strictly feasible traffic allocation schemes in WAN environments, making GRL-TE suitable for deployment in real-world TE systems where both performance and constraint compliance are critical.

4.1 Optimization method for TE

Due to limitations such as link bandwidth and node processing capabilities, the capacity of WANs is inherently constrained, necessitating the use of TE techniques to efficiently allocate resources, prevent congestion, and minimize resource wastage. The list of variables used in this section, along with their descriptions, is shown in Table 1.

The topology of a WAN is represented as a graph $G = (V, E, c)$, where V is the set of network nodes, $n = |V|$ is the total number of nodes in the network, E is the set of physical links, and $c : E \rightarrow \mathbb{R}^+$ represents the capacity of each link. Each traffic demand $d \in D$ represents the traffic from a source node to a destination node, where D is the set of all traffic demands that need to be transmitted in the current time step. The system continuously tracks network status through monitoring modules, periodically evaluating traffic demands

Table 1 List of variables and their descriptions in optimization method for TE

Variable	Description
V	Set of nodes in physical network
E	Set of physical links in physical network
c	Capacity function for each link
$G = (V, E, c)$	WAN topology graph
n	Total number of nodes ($n = V $) in physical network
$c(e)$	Capacity of link e
d	Traffic demand from a source node to a destination node
D	Set of all traffic demands in physical network
P_d	Set of predefined paths for traffic demand d
p	A path in P_d
e	A link in E
$\mathcal{F}$	Traffic allocation
$\mathcal{F}_d(p)$	Proportion of traffic demand d allocated to path p
$\mathcal{F}^t$	Traffic allocation state at time step t

between each pair of nodes for upcoming time steps (Lu et al. 2025), and feeds these into the TE controller.

This paper adopts the path formulation method (Abuzaid et al. 2021), where each traffic demand d is transmitted along a predefined path set P_d . This approach significantly reduces computational complexity by optimizing over a limited path set and is widely adopted in real-world WANs (Saxena et al. 2023; Xu et al. 2023; Krishnaswamy et al. 2022).

The objective of traffic allocation $\mathcal{F}$ is to distribute traffic demands among paths, where $\mathcal{F}_d(p)$ represents the proportion of traffic demand d allocated to path p , ranging from 0 to 1. $\mathcal{F}^t$ represents the traffic allocation state at the t -th time step.

This paper aims to maximize network throughput as the objective function, an approach widely adopted in production TE systems (Hong et al. 2013; Xu et al. 2023; Krishnaswamy et al. 2022). Note that in the mathematical formulations below, d represents both the traffic demand and its corresponding volume.

$$\text{maximize } \sum_{d \in D} \sum_{p \in P_d} \mathcal{F}_d(p) \cdot d \quad (1)$$

The TE model contains three key constraints:

- Demand constraint: For any traffic demand $d \in D$, the total allocation proportion across all paths $p \in P_d$ must not exceed the actual demand.

$$\sum_{p \in P_d} \mathcal{F}_d(p) \leq 1, \quad \forall d \in D \quad (2)$$

- Capacity constraint: For any link $e \in E$ in the network, the load on it must not exceed its capacity.

$$\sum_{d \in D} \sum_{p \in P_d: e \in p} \mathcal{F}_d(p) \cdot d \leq c(e), \quad \forall e \in E \quad (3)$$

- Non-negativity constraint: Ensures that traffic allocation proportions are non-negative, thereby guaranteeing the rationality and physical feasibility of the allocation.

$$\mathcal{F}_d(p) \geq 0, \quad \forall d \in D, \forall p \in P_d \quad (4)$$

By combining the above objectives and constraints, the TE controller can optimize traffic allocation among paths while maximizing network throughput and ensuring network stability.

4.2 TopoFlowNet for feature learning

WAN TE faces challenges in complex network topologies, dynamic traffic demands, and global resource scheduling.

GNNs can effectively model complex dependencies in networks through efficient information-passing mechanisms, thereby achieving dynamic traffic scheduling and optimization.

However, traditional GNNs primarily focus on nodes and edges in graphs, making it difficult to simultaneously express: (1) the containment relationship between paths and links, (2) the collaborative relationship among multiple candidate paths for the same traffic demand, (3) the competitive relationship among different traffic demands on shared links. To address this, we propose the TopoFlowNet structure, which constructs a bipartite graph that models links and paths as two different types of nodes: edge nodes with capacity as features and path nodes with traffic demands as features. This bipartite graph structure naturally models the containment relationship between paths and links. Based on this foundation, the GINConv module implements information-passing between edge nodes and path nodes, allowing the network to perceive resource competition. Finally, the MLP module handles collaborative relationships among multiple candidate paths for the same traffic demand. This hybrid architecture enables the collaborative perception of both capacity and demand constraints, providing rich feature representations for subsequent traffic allocation decisions. The list of variables used in this section, along with their descriptions, is shown in Table 2.

4.2.1 Problem modeling and graph representation

Given a physical network topology $G = (V, E)$, TopoFlowNet constructs a bipartite graph (Tinhofer and Tinhofer 1976) $\mathcal{G} = (\mathcal{V}_e \cup \mathcal{V}_p, \mathcal{E})$, modeling links and paths as two distinct types of nodes. The edge node set $\mathcal{V}_e$ corresponds to the links in the physical network, with $|\mathcal{V}_e| = |E|$, focusing on modeling capacity constraints. The path node set $\mathcal{V}_p$ represents all candidate paths for traffic demands D , with $|\mathcal{V}_p| = \sum_{d \in D} |P_d|$, focusing on modeling demand allocation. The total number of nodes in the bipartite graph is $|\mathcal{V}| = |\mathcal{V}_e| + |\mathcal{V}_p|$. The edge set $\mathcal{E}$ connects only different types of nodes, clearly encoding the containment relationship between paths and links. As shown in Fig. 3, if path $p_j \in \mathcal{V}_p$ passes through link $e_i \in \mathcal{V}_e$, then path node p_j is connected to edge node e_i . This bipartite graph structure naturally transforms the complex task of learning network state representations into an information propagation problem on graphs.

4.2.2 TopoFlowNet network architecture

TopoFlowNet adopts an L -layer stacked hybrid neural network architecture (with layers numbered from 1 to L), where each layer performs one round of information-passing. The first layer captures constraint information from directly adjacent nodes (1-hop neighbors), while the l -th layer aggregates

Table 2 List of variables and their descriptions in TopoFlowNet

Variable	Description
$\mathcal{G} = (\mathcal{V}_e \cup \mathcal{V}_p, \mathcal{E})$	Bipartite graph representation
$\mathcal{V}_e$	Edge node set (physical links)
$\mathcal{V}_p$	Path node set (candidate paths for all traffic demands)
$\mathcal{E}$	Edge set in bipartite graph
$ \mathcal{V} $	Total number of nodes in bipartite graph
e_i	The i -th edge node in $\mathcal{V}_e$
p_j	The j -th path node in $\mathcal{V}_p$
L	Number of layers in TopoFlowNet
l	Layer index (1 to L)
h^0	Initial feature vector
h^l	Output of layer l
$h_{\mathcal{V}_e}^0$	Initial edge features encoding link capacity
$h_{\mathcal{V}_p}^0$	Initial path features encoding traffic demand
d_{p_j}	Traffic demand that path p_j belongs to
$h_{e_i}^l$	Feature of edge node e_i at layer l
$h_{p_j}^l$	Feature of path node p_j at layer l
ϵ^l	Learnable parameter at layer l
$\mathcal{N}(e_i)$	Path nodes adjacent to edge node e_i
$\mathcal{N}(p_j)$	Edge nodes adjacent to path node p_j
W_1^l, W_2^l	GINConv weight matrices at layer l
b_1^l, b_2^l	GINConv bias terms at layer l
h_{GINConv}^l	GINConv module output at layer l
k	Number of candidate paths per demand
h_{reshape}^l	Reshaped path features at layer l
W_3^l, W_4^l	MLP weight matrices at layer l
b_3^l, b_4^l	MLP bias terms at layer l
h_{MLP}^l	MLP module output at layer l
$h_{\mathcal{V}_p}^{l, \text{update}}$	Updated path features at layer l
$h^{l, \text{concat}}$	Concatenated features at layer l
$h_{\mathcal{V}_p}^L$	Final path node embedding representation

information from the l -hop neighborhood, expanding the receptive field. By stacking L -layer, comprehensive network information propagation is achieved.

Feature initialization The initial feature vector $h^0 \in \mathbb{R}^{|\mathcal{V}| \times 1}$ uniformly encodes the network state information as follows:

$$h^0 = \begin{bmatrix} h_{\mathcal{V}_e}^0 \\ h_{\mathcal{V}_p}^0 \end{bmatrix} \quad (5)$$

where the edge node features $h_{\mathcal{V}_e}^0 = [c(e_1), \dots, c(e_i), \dots, c(e_{|\mathcal{V}_e|})]^T$ represent the capacity of each link, and path node features $h_{\mathcal{V}_p}^0 = [d_{p_1}, \dots, d_{p_j}, \dots, d_{p_{|\mathcal{V}_p|}}]^T$ represent

the traffic demands corresponding to each path, where d_{p_j} denotes the size of traffic demand to which path p_j belongs.

Layer-wise feature propagation TopoFlowNet adopts a layer-wise increasing feature dimension design, where the input features of the l -th layer are the output from the previous layer $h^{l-1} \in \mathbb{R}^{|\mathcal{V}| \times l}$, and the output features are $h^l \in \mathbb{R}^{|\mathcal{V}| \times (l+1)}$. The forward propagation of each layer consists of three key steps:

Step 1: GINConv Bidirectional Information-Passing.

The GINConv module performs bidirectional information-passing between edge nodes and path nodes. For an edge node $e_i \in \mathcal{V}_e$, its feature update is given by:

$$h_{e_i}^l = \text{MLP}^l \left((1 + \epsilon^l) \cdot h_{e_i}^{l-1} + \sum_{p_j \in \mathcal{N}(e_i)} h_{p_j}^{l-1} \right) \quad (6)$$

where $h_{e_i}^{l-1}$ is the feature from the previous layer, ϵ^l is a learnable parameter, and $\mathcal{N}(e_i)$ represents the set of path nodes adjacent to edge node e_i .

For a path node $p_j \in \mathcal{V}_p$, its feature update is given by:

$$h_{p_j}^l = \text{MLP}^l \left((1 + \epsilon^l) \cdot h_{p_j}^{l-1} + \sum_{e_i \in \mathcal{N}(p_j)} h_{e_i}^{l-1} \right) \quad (7)$$

where $\mathcal{N}(p_j)$ is the set of edge nodes adjacent to path node p_j .

The MLP structure is defined as:

$$\text{MLP}^l(x) = \text{ReLU}(x \cdot W_1^l + b_1^l) \cdot W_2^l + b_2^l \quad (8)$$

where $W_1^l, W_2^l \in \mathbb{R}^{l \times l}$, $b_1^l, b_2^l \in \mathbb{R}^l$. The final feature update is represented as:

$$h_{\text{GINConv}}^l = \begin{bmatrix} h_{\mathcal{V}_e}^l \\ h_{\mathcal{V}_p}^l \end{bmatrix} \quad (9)$$

$$\text{where } h_{\mathcal{V}_e}^l = \begin{bmatrix} h_{e_1}^l \\ \vdots \\ h_{e_{|\mathcal{V}_e|}}^l \end{bmatrix}, h_{\mathcal{V}_p}^l = \begin{bmatrix} h_{p_1}^l \\ \vdots \\ h_{p_{|\mathcal{V}_p|}}^l \end{bmatrix}.$$

Through the GINConv module's information aggregation, edge nodes can perceive traffic demand information from all paths passing through them, while path nodes can aggregate capacity states from all links they traverse. This bidirectional information-passing mechanism provides the model with a structural foundation to perceive both capacity and demand constraints, enabling subsequent network layers to learn congestion assessment and path availability judgment.

Step 2: MLP Path Collaborative Decision.

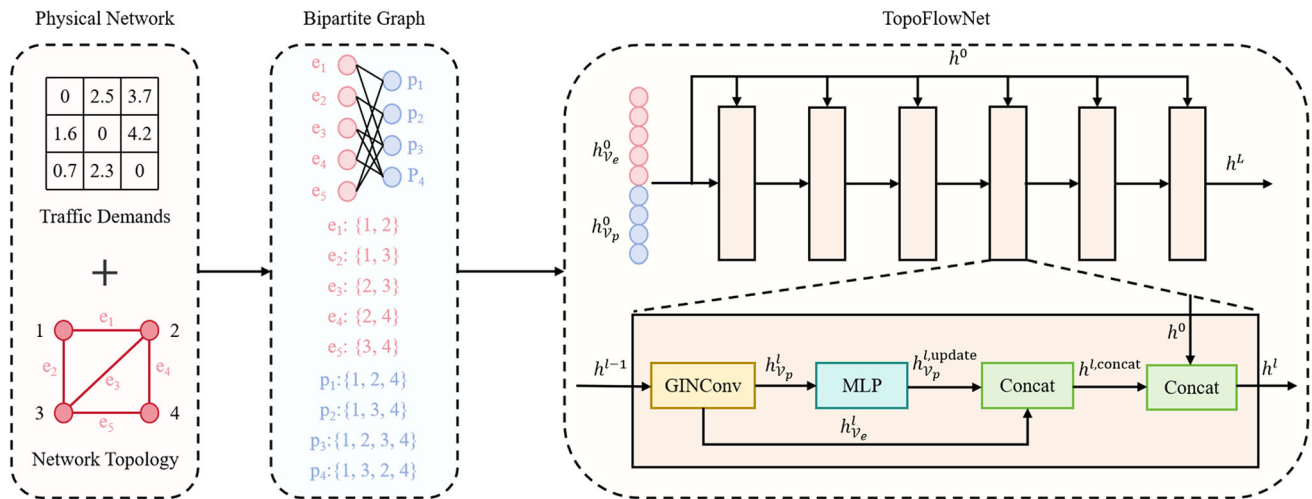


Fig. 3 Framework of TopoFlowNet architecture

From h^l_{GINConv} , extract the path node features $h^l_{\mathcal{V}_p} \in \mathbb{R}^{|\mathcal{V}_p| \times l}$, and then perform reshaping. Assuming each traffic demand has k candidate paths, with a total of $|\mathcal{V}_p|/k$ traffic demands, reshape the path node features into the form $\mathbb{R}^{(|\mathcal{V}_p|/k) \times (kl)}$, organizing all path features belonging to the same traffic demand together: $h^l_{\text{reshape}} = \text{Reshape}(h^l_{\mathcal{V}_p})$. This reshaping operation ensures that path features belonging to the same traffic demand are adjacent in the feature dimension, facilitating MLP learning of relative relationships among paths. Then, input the reshaped features into the MLP layer for nonlinear transformation:

$$h^l_{\text{MLP}} = \text{MLP}(h^l_{\text{reshape}}) = \text{ReLU}(h^l_{\text{reshape}} \cdot W_3^l + b_3^l) \cdot W_4^l + b_4^l \quad (10)$$

where $W_3^l, W_4^l \in \mathbb{R}^{(kl) \times (kl)}$ and $b_3^l, b_4^l \in \mathbb{R}^{kl}$. Finally, reshape the processed features back to the original shape to obtain updated path features $h^{l,\text{update}}_{\mathcal{V}_p} \in \mathbb{R}^{|\mathcal{V}_p| \times l}$.

By organizing all candidate path features for the same traffic demand together, the MLP module can learn and identify path state semantics (such as congestion levels and available capacity) based on feature information aggregated by GINConv, enabling each path node to obtain enhanced feature representations that consider other paths of the same demand.

Step 3: Feature Concatenation and Skip Connection.

After processing by GINConv and MLP, the features from two different perspectives need to be integrated. The GINConv output contains fused capacity and demand information from bidirectional information passing, while the MLP output contains collaborative path state features. Therefore, concatenate the edge node features processed by GINConv with the path node features processed by MLP along the node

dimension:

$$h^{l,\text{concat}} = \text{concat}(h^l_{\mathcal{V}_e}, h^{l,\text{update}}_{\mathcal{V}_p}) \in \mathbb{R}^{|\mathcal{V}| \times l} \quad (11)$$

To enhance information flow stability and accelerate training, TopoFlowNet uses skip connections in each layer. Skip connections concatenate the current layer's output with the initial feature embedding h^0 along the feature dimension, ensuring the model retains important initial information:

$$h^l = \text{concat}(h^{l,\text{concat}}, h^0) \in \mathbb{R}^{|\mathcal{V}| \times (l+1)} \quad (12)$$

Thus, the final output feature dimension of the l -th layer becomes $l + 1$. Through this incremental feature dimension design, the network can gradually learn higher-level abstract representations while retaining basic information.

Output layer Since traffic allocation decisions are made at the path level, after L layers of feature propagation, the final output returns the embedding representation of path nodes $h^L_{\mathcal{V}_p} \in \mathbb{R}^{|\mathcal{V}_p| \times (L+1)}$. The $(L + 1)$ -dimensional features contain L dimensions of learned features and one dimension of initial feature information, providing rich feature representations for subsequent RL policies.

4.3 DRL-based traffic allocation

The core idea of the A2C algorithm is to combine the policy network (Actor) with the value network (Critic), which helps to reduce the variance of policy gradients through advantage estimation, thereby improving the stability of training. Unlike value-based methods (e.g., DQN), A2C naturally supports continuous action spaces, enabling precise control over traffic allocation ratios across paths—an essential requirement in

TE. Compared to more complex algorithms, such as PPO or SAC, A2C offers an optimal trade-off between performance and computational efficiency. Its simpler architecture results in lower computational overhead, which is crucial for real-time TE decisions. In the GRL-TE framework, we integrate A2C with TopoFlowNet to generate high-quality initial traffic allocations by combining structured feature learning with RL. The list of variables used in this section, along with their descriptions, is shown in Table 3.

4.3.1 Integration architecture

Network state representation The A2C algorithm reorganizes the feature representations $h_{\mathcal{V}_p}^L$ from TopoFlowNet based on traffic demand:

$$h_{\text{reshape}} = \text{Reshape}(h_{\mathcal{V}_p}^L) \in \mathbb{R}^{(|\mathcal{V}_p|/k) \times (k(L+1))} \quad (13)$$

where $|\mathcal{V}_p|/k = |D|$ represents the total number of network traffic demands.

Policy network architecture The policy network generates the Gaussian distribution parameters through two independent linear layers:

$$\mu_{\theta}(s_t) = h_{\text{reshape}} \cdot W_{\mu} + b_{\mu} \in \mathbb{R}^{|D| \times k} \quad (14)$$

$$\log \sigma_{\theta}(s_t) = h_{\text{reshape}} \cdot W_{\sigma} + b_{\sigma} \in \mathbb{R}^{|D| \times k} \quad (15)$$

where $W_{\mu}, W_{\sigma} \in \mathbb{R}^{(k(L+1)) \times k}$ are learnable weight matrices, and $b_{\mu}, b_{\sigma} \in \mathbb{R}^{|D| \times k}$ are learnable bias terms.

Action generation process The policy $\pi_{\theta}(a_t|s_t)$ parameterized by θ is modeled as a Gaussian distribution:

$$\pi_{\theta}(a_t|s_t) \sim \mathcal{N}(\mu_{\theta}(s_t), \sigma_{\theta}^2(s_t)) \quad (16)$$

The variance is computed as:

$$\sigma_{\theta}^2(s_t) = (\exp(\log \sigma_{\theta}(s_t)))^2 \quad (17)$$

During training, we sample raw actions from this distribution:

$$a_t^{\text{raw}} \sim \mathcal{N}(\mu_{\theta}(s_t), \sigma_{\theta}^2(s_t)) \quad (18)$$

where $a_t^{\text{raw}} \in \mathbb{R}^{|D| \times k}$ represents the traffic allocation for all demands across their candidate paths.

Next, we apply softmax normalization to each traffic demand's raw actions to obtain allocation proportions on candidate paths. For the m -th traffic demand d_m , the normalized

Table 3 List of variables and their descriptions in DRL

Variable	Description
$h_{\mathcal{V}_p}^L$	Path node embedding representation from TopoFlowNet
h_{reshape}	Reshaped feature representation organizing path features by traffic demand
s_t	State at time step t
s_{t+1}	State at time step $t + 1$
a_t	Action (traffic allocation) at time step t
θ	Policy network parameters
w	Value network parameters
$\pi_{\theta}(a_t s_t)$	Parameterized policy
$\mu_{\theta}(s_t)$	Mean output of policy network
$\log \sigma_{\theta}(s_t)$	Log standard deviation output of policy network
$\sigma_{\theta}^2(s_t)$	Variance of policy distribution
W_{μ}, W_{σ}	Policy network weight matrices
b_{μ}, b_{σ}	Policy network bias terms
a_t^{raw}	Raw action sampled from Gaussian distribution
d_m	The m -th traffic demand
$\mathcal{F}_{d_m}(p)$	Proportion of demand d_m allocated to path p
$a_{m,p}$	Traffic allocation for demand m on path p
m	Demand index (1 to $ D $)
p	Path index (1 to k)
P_{d_m}	Set of candidate paths for demand d_m
ϵ	Small tolerance value for numerical stability
ratio_m	Allocation ratio for demand m
$\tilde{a}_{m,p}$	Allocation after demand constraint adjustment
u_e	Utilization of edge e
$\hat{a}_{m,p}$	Allocation after capacity constraint adjustment
F_{total}	Total network allocation after constraints
$R(s_t, a_t)$	Reward function
r_t	Immediate reward
$V(s_t; w)$	State value function
$\mathbb{E}_{\pi_{\theta}}[\cdot]$	Expectation under policy π_{θ}
γ	Discount factor (standard A2C)
τ	Future time step index
δ_t	Temporal difference error
α	Policy network learning rate
β	Value network learning rate
$\nabla_{\theta} \log \pi_{\theta}(a_t s_t)$	Policy gradient
$\nabla_w V(s_t; w)$	Value function gradient
$\Delta \theta$	Policy network parameter update
Δw	Value network parameter update
$A(s_t, a_t)$	Advantage function

traffic allocation is given by:

$$\mathcal{F}_{d_m}(p) = \frac{\exp(a_{m,p}^{\text{raw}})}{\sum_{p'=1}^k \exp(a_{m,p'}^{\text{raw}})} \quad (19)$$

where $m = 1, \dots, |D|$, $p = 1, \dots, k$, and $\mathcal{F}_{d_m}(p)$ represents the proportion of demand d_m allocated to the p -th path. This allocation satisfies the constraint:

$$\sum_{p=1}^k \mathcal{F}_{d_m}(p) = 1 \quad (20)$$

The final traffic allocation for demand d_m on path p is:

$$a_{m,p} = \mathcal{F}_{d_m}(p) \cdot d_m \quad (21)$$

Thus, the complete traffic allocation $a_t \in \mathbb{R}^{|D| \times k}$ for all demands across all paths is represented as:

$$a_t = \{a_{m,p}\}, \quad \forall m \in \{1, \dots, |D|\}, p \in \{1, \dots, k\} \quad (22)$$

During deployment, to ensure stable traffic allocation, we use the mean output $\mu_\theta(s_t)$ of the policy network and apply softmax normalization (19) to obtain deterministic traffic allocation proportions, thereby eliminating the need for random sampling from the Gaussian distribution.

Constraint-aware action adjustment and reward calculation

The traffic allocation action a_t generated by the policy network during training, or refined by ADMM with fixed iterations during deployment, need to undergo further constraint adjustments to ensure strict network feasibility. This adjustment mechanism serves dual purposes: during the training phase, it ensures feasible allocations for reward calculation, while in the deployment phase following ADMM optimization, it guarantees that the final solution strictly satisfies all network constraints.

Step 1: Demand Constraint Validation and Adjustment.

Due to numerical precision in computation, we verify that the total allocation for each demand $d_m \in D$ does not exceed the demand value. For numerical stability, if:

$$\sum_{p=1}^k a_{m,p} > d_m + \epsilon \quad (23)$$

where ϵ is a small tolerance value, we perform proportional adjustment. Firstly, we calculate the ratio ratio_m of the total allocation to the demand:

$$\text{ratio}_m = \frac{\sum_{p=1}^k a_{m,p}}{d_m} \quad (24)$$

If $\text{ratio}_m > 1$, it indicates that the allocation exceeds the demand. In this case, we reduce the allocation on each path proportionally to ensure the adjusted allocation does not

exceed the demand. The adjustment formula is:

$$\tilde{a}_{m,p} = \frac{a_{m,p}}{\text{ratio}_m} \quad \forall p \in \{1, 2, \dots, k\} \quad (25)$$

This operation ensures that the total allocation for each demand satisfies the following constraint:

$$\sum_{p=1}^k \tilde{a}_{m,p} \leq d_m \quad (26)$$

Step 2: Capacity Constraint Validation and Adjustment.

Once the demand constraints are satisfied, we address the capacity constraints. If the allocation on some paths exceeds the network's capacity limit, we adjust the allocation according to the capacity. For each edge $e \in E$, we first calculate its utilization:

$$u_e = \frac{\sum_{m=1}^{|D|} \sum_{p \in P_{d_m}: e \in p} \tilde{a}_{m,p}}{c(e)} \quad (27)$$

If the utilization $u_e > 1$, the allocation on the relevant paths needs to be proportionally reduced to ensure it does not exceed the capacity. The adjustment formula is:

$$\hat{a}_{m,p} = \frac{\tilde{a}_{m,p}}{\max_{e \in p} \{u_e, 1\}}, \quad \forall m \in \{1, \dots, |D|\}, p \in \{1, \dots, k\} \quad (28)$$

Step 3: Allocation Adjustment and Reward Calculation. The total network allocation after constraint adjustment F_{total} is defined as:

$$F_{\text{total}} = \sum_{m=1}^{|D|} \sum_{p=1}^k \hat{a}_{m,p} \quad (29)$$

This total allocation serves as the reward r_t in the A2C framework:

$$r_t = R(s_t, a_t) = F_{\text{total}} \quad (30)$$

Value network design The value network utilizes the reorganized feature representations for value estimation:

$$V(s_t; w) = \text{Critic}(h_{\text{reshape}}; w) \quad (31)$$

End-to-end optimization The shared TopoFlowNet architecture enables efficient end-to-end optimization of the entire framework. Both the policy and value networks leverage the same TopoFlowNet module for feature extraction, allowing

reward signals to propagate through gradients and back-propagate to all parameters in a coordinated manner. This deep integration ensures that the learned graph features are specifically tailored for the traffic allocation task, providing a high-quality initial traffic allocation scheme and a solid starting point for subsequent ADMM optimization.

4.3.2 Standard A2C mechanism

Standard A2C relies on multi-step cumulative returns and future state predictions.

Value estimation and temporal difference error The value network estimates the expected cumulative discounted return $V(s_t; w)$ starting from the current state s_t :

$$V(s_t; w) = \mathbb{E}_{\pi_\theta} \left[\sum_{\tau=0}^{\infty} \gamma^\tau r_{t+\tau} \mid s_t \right] \quad (32)$$

where w denotes the value network parameters, $\gamma \in (0, 1]$ is the discount factor, τ is the future time step index, $r_{t+\tau}$ is the immediate reward obtained after taking action at step $t + \tau$, and $\mathbb{E}_{\pi_\theta}[\cdot]$ denotes the expectation under the policy π_θ .

The temporal difference (TD) error measures the discrepancy between the current value estimate and the target. For clarity, we focus on the one-step TD variant, which serves as a baseline for our one-step reward mechanism:

$$\delta_t = r_t + \gamma V(s_{t+1}; w) - V(s_t; w) \quad (33)$$

where r_t is the immediate reward obtained after executing action a_t in state s_t , and $V(s_{t+1}; w)$ is the value estimate of the next state.

Parameter updates The policy network and value network are updated using the TD error.

$$\Delta\theta = \alpha \cdot \delta_t \cdot \nabla_\theta \log \pi_\theta(a_t | s_t) \quad (34)$$

$$\Delta w = \beta \cdot \delta_t \cdot \nabla_w V(s_t; w) \quad (35)$$

where $\Delta\theta$ and Δw represent weight updates, $\alpha > 0$ and $\beta > 0$ are the learning rates for the policy network and value network respectively, $\nabla_\theta \log \pi_\theta(a_t | s_t)$ is the gradient of the log probability of policy π_θ concerning parameters θ , and $\nabla_w V(s_t; w)$ is the gradient of the value function $V(s_t; w)$ concerning parameters w .

Figure 4 shows the architecture of standard A2C. The value network requires both the current state s_t and the next state s_{t+1} to compute the TD error δ_t . This process helps in updating the policy and value network to optimize the agent's performance in future steps.

4.3.3 One-step A2C mechanism

In WAN scenarios, traffic demands are independently given at each time step, and current allocation decisions do not influence future traffic demands. Based on this characteristic, we design a one-step reward mechanism to extend the A2C framework, focusing training on immediate rewards.

One-step A2C uses the same action generation process as described in Section 4.3.1 (14–21), but with modifications in value estimation and updates:

Immediate value estimation The state value in one-step A2C estimates only the expected immediate return:

$$V(s_t; w) = \mathbb{E}_{\pi_\theta} [r_t \mid s_t] \quad (36)$$

where $r_t = R(s_t, a_t)$ represents the global total traffic reward after executing action a_t in state s_t , i.e., the immediate reward, and $R(\cdot, \cdot)$ is the reward function.

The advantage function is defined as:

$$A(s_t, a_t) = r_t - V(s_t; w) \quad (37)$$

which represents the difference between the immediate reward and the value estimate of the current state.

Simplified parameter updates Both the policy network and value network are updated using the advantage function. The

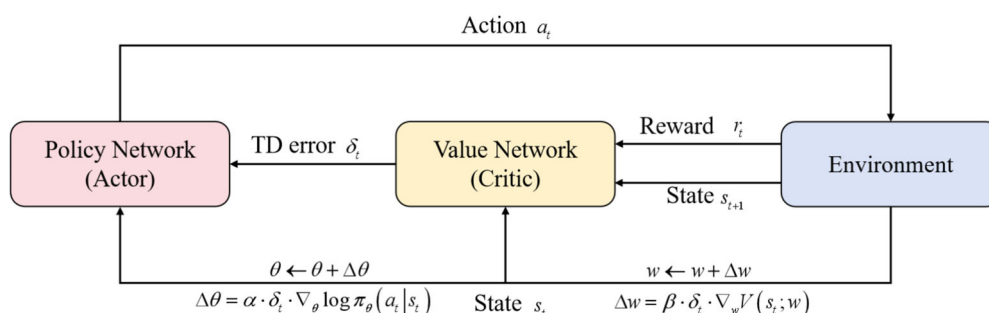


Fig. 4 Standard A2C architecture: parameter updates using TD error

parameter updates are given by:

$$\Delta\theta = \alpha \cdot A(s_t, a_t) \cdot \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \quad (38)$$

$$\Delta w = \beta \cdot A(s_t, a_t) \cdot \nabla_w V(s_t; w) \quad (39)$$

where $\Delta\theta$ and Δw represent the update quantities for the policy and value networks, respectively. The policy network uses advantage-weighted updates to improve the policy, increasing probabilities for better-than-expected actions and decreasing them for worse-than-expected actions. The value network updates through the advantage function gradient to reduce prediction error, making the value estimate gradually approach the observed immediate reward.

Figure 5 shows the simplified architecture of one-step A2C, where the advantage function is directly computed from the difference between the immediate reward r_t and the value estimate $V(s_t; w)$.

This design eliminates the need for the discount factor γ and future state estimation, significantly simplifying the training objective while preserving A2C's core advantage-based learning mechanism.

4.4 ADMM for TE optimization

Although DRL-based methods can efficiently generate initial solutions and provide rapid traffic allocation strategies, they face inherent limitations: neural networks cannot guarantee strict constraint satisfaction (e.g., link capacity constraints), which may result in infeasible solutions. To address this challenge, we introduce the ADMM as a refinement step following the DRL allocation. ADMM is a highly parallelizable optimization algorithm that enables fast computation, making it particularly suitable for real-time TE applications. Through its iterative optimization process, ADMM systematically mitigates constraint violations while improving solution quality. The list of variables used in this section, along with their descriptions, is shown in Table 4.

4.4.1 Problem transformation

First, we transform the objective function and constraints into a form suitable for the ADMM method. The objective is to maximize network throughput, as shown in (1).

The original problem contains three types of constraints: (1) Demand constraints as shown in (2). (2) Capacity constraints as shown in (3). (3) Non-negativity constraints as shown in (4).

To adapt to ADMM's block optimization structure, we reorganize the constraints as: (1) Demand constraint G_{1d} : maintains the original demand allocation constraint; (2) Capacity constraint G_{3e} : maintains the original link capacity constraint; (3) Flow consistency constraint G_{4dpe} : newly introduced constraint to ensure consistency between path flows and link flows.

The non-negativity constraints are handled differently in different stages. In the DRL stage, the allocation proportions generated through softmax naturally satisfy non-negativity. In the ADMM optimization stage, we ensure non-negativity of link flows and path flows through ReLU projection operations.

4.4.2 Decoupling path flows and link flows

In the original problem, path-level decision variables $\mathcal{F}_d(p)$ and link-level capacity constraints $c(e)$ are coupled. Specifically, a path p may traverse multiple links, and each link e may have flows from multiple paths passing through it. This many-to-many relationship creates tight coupling between path flows and link flows, making the problem difficult to solve.

To decouple this complex relationship, we introduce auxiliary variables z_{dpe} , representing the flow of demand d through path p on link e . Thus, the original capacity

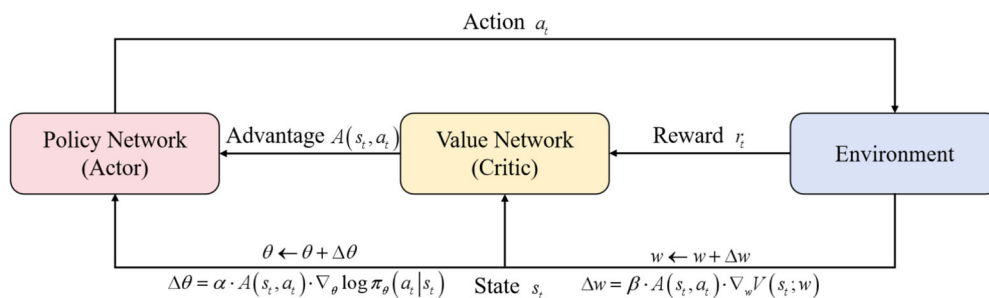


Fig. 5 One-step A2C architecture: computing advantage function using immediate reward

Table 4 List of variables and their descriptions in ADMM

Variable	Description
z_{dpe}	Auxiliary variable: flow of demand d through path p on link e
ξ_{1d}	Slack variable for demand constraint (unallocated portion of demand d)
ξ_{3e}	Slack variable for capacity constraint (remaining capacity of link e)
ξ	Vector of all slack variables
z	Vector of all auxiliary variables
$G(\mathcal{F}, z, \xi)$	Set of constraint functions
G_{1d}	Demand constraint function for demand d
G_{3e}	Capacity constraint function for link e
G_{4dpe}	Flow consistency constraint function
$\mathcal{L}_p(\mathcal{F}, z, \xi, \lambda)$	Augmented Lagrangian function
λ	Lagrange multipliers
ρ	Penalty parameter
k	Iteration index
$\mathcal{F}^k$	Path flow allocation at iteration k
z^k	Auxiliary variables at iteration k
ξ^k	Slack variables at iteration k
λ^k	Lagrange multipliers at iteration k
$\mathcal{F}^{k+1}$	Updated path flow allocation
z^{k+1}	Updated auxiliary variables
ξ^{k+1}	Updated slack variables
λ^{k+1}	Updated Lagrange multipliers

constraint (3) is decomposed into two constraints:

$$\sum_{d \in D} \sum_{p \in P_d: e \in p} z_{dpe} \leq c(e), \quad \forall e \in E \quad (40)$$

$$\mathcal{F}_d(p) \cdot d - z_{dpe} = 0, \quad \forall d \in D, \quad p \in P_d, \quad e \in p \quad (41)$$

where (40) represents the link capacity constraint and (41) ensures flow consistency.

4.4.3 Introducing slack variables

ADMM prefers to handle equality constraints. Therefore, by introducing slack variables ξ_{1d} and ξ_{3e} , we convert inequality constraints to equality constraints. $\xi_{1d} \geq 0$ represents the unallocated portion of demand d , allowing demand constraints to be flexibly adjusted during the optimization process; $\xi_{3e} \geq 0$ represents the remaining capacity of link e , decoupling capacity constraints from path flow variables. During the iteration process, the values of slack variables directly reflect the degree to which the current solution satisfies the constraints. By introducing slack variables, the

original constraints are rewritten as:

$$\sum_{p \in P_d} \mathcal{F}_d(p) + \xi_{1d} - 1 = 0, \quad \forall d \in D \quad (42)$$

$$\sum_{d \in D} \sum_{p \in P_d: e \in p} z_{dpe} + \xi_{3e} - c(e) = 0, \quad \forall e \in E \quad (43)$$

4.4.4 Constructing the augmented Lagrangian function

The key to the ADMM method is constructing the augmented Lagrangian function, which combines the objective function with all constraints. The set of constraint functions is defined as follows:

$$G(\mathcal{F}, z, \xi) = \begin{bmatrix} G_{1d} \\ G_{3e} \\ G_{4dpe} \end{bmatrix} \quad (44)$$

where:

$$G_{1d} = \sum_{p \in P_d} \mathcal{F}_d(p) + \xi_{1d} - 1 \quad (45)$$

$$G_{3e} = \sum_{d \in D} \sum_{p \in P_d: e \in p} z_{dpe} + \xi_{3e} - c(e) \quad (46)$$

$$G_{4dpe} = \mathcal{F}_d(p) \cdot d - z_{dpe} \quad (47)$$

The augmented Lagrangian function is expressed as:

$$\mathcal{L}_p(\mathcal{F}, z, \xi, \lambda) = -\sum_{d \in D} \sum_{p \in P_d} \mathcal{F}_d(p) \cdot d + \lambda^T G(\mathcal{F}, z, \xi) + \frac{\rho}{2} \|G(\mathcal{F}, z, \xi)\|_2^2 \quad (48)$$

where $\sum_{d \in D} \sum_{p \in P_d} \mathcal{F}_d(p) \cdot d$ represents the original objective function. $\lambda^T G(\mathcal{F}, z, \xi)$ is the Lagrangian multiplier term, where λ is the Lagrange multiplier. $\frac{\rho}{2} \|G(\mathcal{F}, z, \xi)\|_2^2$ is the quadratic penalty term, with ρ as the penalty parameter.

4.4.5 Iterative optimization process

The core of the ADMM method is iteratively optimizing path flows, auxiliary variables, and slack variables while adjusting Lagrange multipliers to gradually approach a feasible solution and obtain the final traffic allocation strategy. In each iteration, variables are optimized in the following order:

Step 1: Update path flows:

$$\mathcal{F}^{k+1} = \arg \min_{\mathcal{F}} \mathcal{L}_p(\mathcal{F}, z^k, \xi^k, \lambda^k) \quad (49)$$

Step 2: Update auxiliary variables:

$$z^{k+1} = \arg \min_z \mathcal{L}_p(\mathcal{F}^{k+1}, z, \xi^k, \lambda^k) \quad (50)$$

Step 3: Update slack variables:

$$\xi^{k+1} = \arg \min_{\xi} \mathcal{L}_p(\mathcal{F}^{k+1}, z^{k+1}, \xi, \lambda^k) \quad (51)$$

Step 4: Update Lagrange multipliers:

$$\lambda^{k+1} = \lambda^k + \rho \cdot G(\mathcal{F}^{k+1}, z^{k+1}, \xi^{k+1}) \quad (52)$$

The initial flow allocation strategy is generated through the existing policy network (warm start).

5 Evaluation

5.1 Dataset

We used the following WAN datasets, listed in order of increasing network size:

- B4 (Jain et al. 2013): Google's private WAN, designed specifically for high-bandwidth communication needs between its global data centers. This network uses SDN technology to implement centralized TE, serving as a typical example of SDN-TE in the industry.
- GÉANT (Uhlir et al. 2006): The backbone topology of the European research and education network, connecting national research and education networks across Europe. As one of the largest research and education networks in the world, it provides high-speed international connectivity for European research institutions.
- UsCarrier (Knight et al. 2011): The backbone network topology of a U.S. carrier, from the Internet Topology Zoo, reflecting the typical structure of a regional commercial Internet Service Provider network.
- Cogentco (Knight et al. 2011): The global backbone network topology of Cogent Communications. As a Tier-1 internet service provider, its network covers major cities in North America, Europe, and Asia.

- Kdl (Knight et al. 2011): The national backbone network topology of Korea Telecom, illustrating the large-scale network architecture of a national telecom operator.
- ASN (CAIDA 2022): An Autonomous System (AS)-level topology based on the Center for Applied Internet Data Analysis AS relationship dataset. While the original data reflects inter-domain routing relationships, it has been adapted for simulating large-scale WAN TE scenarios.

Table 5 shows the detailed information for each topology. These datasets cover network sizes ranging from small-scale private WANs (B4, 12 nodes) to large-scale AS-level topologies (ASN, 1,739 nodes). Notably, although the average shortest path length and network diameter typically increase as the network size grows, the ASN topology exhibits an anomaly: despite having the most nodes, its average path length and diameter are relatively small. This is due to the highly interconnected star-like cluster structure in ASN, which provides strong cluster-level connectivity.

Regarding traffic data, the traffic matrices and link capacities for B4, UsCarrier, Kdl, and ASN are sourced from Teal (Xu et al. 2023). For GÉANT, the data is provided by Figret (Liu et al. 2024). For Cogentco, due to the lack of publicly available traffic data, we generate synthetic traffic using the gravity model (Applegate and Cohen 2003; Roughan et al. 2002; Guo et al. 2025; Liu et al. 2024), which is a classic approach in network traffic modeling. Additionally, for each node pair, we precompute 4 shortest paths as candidate paths for traffic allocation.

5.2 Baseline

We compare the proposed GRL-TE with several existing TE schemes:

- LP: The global LP method directly solves the full TE optimization problem. While this method can achieve the theoretical optimal solution, its high computational complexity may pose scalability issues in large-scale networks.

Table 5 Network characteristics of six WAN topologies

Topology	Nodes	Edges	Average Shortest Path Length	Network Diameter
B4	12	38	2.3	5
GÉANT	22	72	2.53	5
UsCarrier	158	378	12.1	35
Cogentco	197	486	10.51	28
Kdl	754	1,790	22.7	58
ASN	1,739	8,558	3.2	8

- **NCFlow** (Abuzaid et al. 2021): NCFlow divides the network into multiple clusters, solving TE subproblems within each cluster using LP, and then combines the results to generate the global solution.
- **Teal** (Xu et al. 2023): Teal combines MARL with ADMM to optimize TE. It uses FlowGNN to extract features, a policy network for traffic allocation, and ADMM for refining solutions to maximize throughput.
- **Figret** (Liu et al. 2024): Figret is a fine-grained robustness-enhanced TE scheme that provides customized robustness enhancements for different source-destination pairs to handle traffic bursts.

5.3 Experiment setup

GRL-TE operates as follows: In each time step, it receives the WAN topology, link capacity information, a TM containing demands between each pair of nodes, and a precomputed set of 4 candidate paths for each node pair. It outputs the traffic allocation across these 4 paths for each node pair. The core modules of GRL-TE are implemented using the PyTorch framework.

- **TopoFlowNet and A2C Architecture:** TopoFlowNet consists of 6 stacked layers of hybrid neural networks with an output dimension of 7. The A2C algorithm transforms TopoFlowNet's output dimension to 28. The Actor network includes two output heads: a mean head that generates action means via a linear layer with dimensions (28, 4), and a standard deviation head that generates the log standard deviations through another linear layer, also with dimensions (28, 4). The Critic network performs value estimation via a three-layer fully connected network: the first layer has dimensions (28, 256), the second layer has dimensions (256, 256), and the output layer has dimensions (256, 1), with ReLU activation functions applied in the first two layers.
- **ADMM Optimization:** The number of ADMM iterations is adjusted based on the scale of the network topology. For example, the B4 topology uses 2 iterations, UsCarrier, Cogentco, Kdl, and ASN each use 25 iterations, and the GÉANT topology uses 100 iterations due to its significantly larger traffic demands. The penalty factor ρ is set to 1.0, controlling the penalty strength for constraint violations during the ADMM optimization process.

The training process runs for 3 episodes, with the Adam optimizer (Diederik 2014) used for optimization. The learning rate for the Actor is set to 1×10^{-4} , while the Critic's learning rate is set to 1×10^{-3} . All experiments are repeated 10 times, with random seeds set to [100, 200, ..., 1000].

For the baseline model NCFlow, we use the cluster numbers specified in the original paper (36 and 81) for the UsCarrier and Kdl datasets, respectively. The default partitioning method (FMPartitioning) is used for the B4, GÉANT, Cogentco, and ASN datasets.

For the baseline model Teal, we use the ADMM iteration settings from the original paper where available (B4: 2 iterations; UsCarrier, Kdl, ASN: 25 iterations). For datasets not covered in the original paper (Cogentco and GÉANT), we match our GRL-TE settings (25 and 100 iterations respectively) for consistency.

All experiments were conducted on a system equipped with Nvidia GeForce RTX 4090 GPUs, ensuring computational efficiency and resource optimization. The server features an Intel Xeon Silver 4310 processor with 12 cores running at 2.1 GHz, 32 GB of DDR4 RAM (3200 MHz) with error-correcting code support, and four Nvidia GeForce RTX 4090 GPUs optimized for high-performance computing tasks.

5.4 Comparing GRL-TE to the state of the art

We comprehensively evaluated the performance of GRL-TE against four baseline methods on six WAN topologies of different scales. The experimental results of the average demand satisfaction rate are shown in Fig. 6.

Table 6 shows the detailed model performance comparison. Across the six topologies, GRL-TE achieved an overall average demand satisfaction rate of 89.36%, outperforming all learning-based methods, including Figret (82.20%, average of 5 topologies) and Teal (82.04%). Compared to optimization-based methods, GRL-TE also demonstrated competitiveness, outperforming NCFlow (76.48%, a clustering-based optimization method) and ranking second only to the LP solver (93.19%). While LP achieves theo-

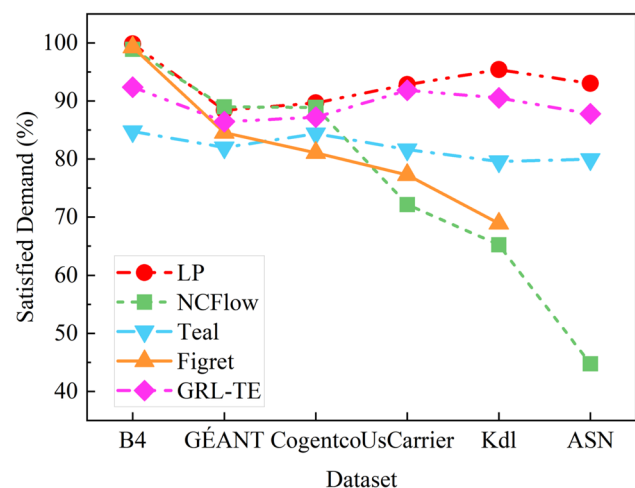


Fig. 6 Total satisfied demand

Table 6 Model performance comparison

Dataset	LP	NCFlow	Teal	Figret	GRL-TE
B4	99.83	98.90	84.77	99.24	92.39
GÉANT	88.40	88.99	81.98	84.53	86.41
UsCarrier	92.83	72.15	81.62	77.27	91.82
Cogentco	89.65	88.87	84.38	81.09	87.20
Kdl	95.40	65.21	79.57	68.89	90.54
ASN	93.02	44.78	79.94	–	87.77
Avg	93.19	76.48	82.04	82.20	89.36
Std	4.11	19.82	2.17	11.17	2.55
Avg Rank	1.17	3.33	3.83	4.00	2.67

retical optimal values, its computational complexity makes it impractical for large-scale network applications, whereas GRL-TE provides the best balance between performance and efficiency.

Small-scale network

- B4: NCFlow (98.9%) and Figret (99.24%) performed exceptionally well, even approaching LP (99.83%). GRL-TE achieved 92.39%, and while not optimal in this scenario, it still significantly outperformed Teal (84.77%).
- GÉANT: NCFlow slightly exceeded LP with 88.99% performance compared to LP's 88.4%, demonstrating its advantage in specific small-scale networks. GRL-TE (86.41%) showed moderate performance on this topology. Notably, GÉANT's average total traffic demand reaches 43.94 billion, which is 1.57 million times that of B4 (27.98 thousand). This extreme traffic intensity leads to a decrease in the performance of all methods.

Medium-scale networks

- UsCarrier: GRL-TE (91.82%) demonstrated clear advantages, approaching LP (92.83%), while NCFlow performance dropped significantly to 72.15%.
- Cogentco: Three methods showed similar performance: NCFlow (88.87%), GRL-TE (87.2%), and LP (89.65%), indicating intense competition in medium-scale synthetic traffic scenarios.

Large-scale networks

- Kdl: GRL-TE (90.54%) maintained high performance, while NCFlow dropped to 65.21% and Figret to 68.89%.
- ASN: GRL-TE (87.77%) was the only learning method that maintained a stable performance in ultra-large-scale networks. At the same time, NCFlow collapsed to

44.78%, and Figret could not complete the experiment due to computational resource limitations.

In evaluating TE, we found significant scale dependency in the performance of different methods. NCFlow performed excellently in small-scale networks, but its performance degraded sharply as network scale increased, especially in large-scale networks where its clustering-based optimization method failed significantly, with performance dropping by up to 54.12 percentage points. In contrast, GRL-TE maintained stable performance across networks of various scales (86.41%–92.39%), demonstrating excellent scalability, particularly when scaling from B4 to ASN (145x increase in nodes), with performance dropping by only 4.62 percentage points.

Furthermore, GRL-TE's performance stability outperformed most methods, with a standard deviation of 2.55%, significantly lower than NCFlow's 19.82% and Figret's 11.17%, indicating that these two methods are susceptible to network characteristics. Although Teal showed good stability (standard deviation 2.17%), its overall performance was low. In production environments, especially for medium- to large-scale networks, GRL-TE is the only learning method capable of maintaining an approximately 90% demand satisfaction rate. Although not optimal in some small-scale scenarios, it provides the best “all-scenario” solution.

Following references (Rusdi et al. 2023; Jamaludin et al. 2022), we conducted a Friedman test to validate the superiority of our proposed method across six network topologies. The test revealed significant differences among the five algorithms ($\chi^2 = 12.667, df = 4, p < 0.05$). Post-hoc Nemenyi test showed that while LP achieved the best average rank (1.17), our proposed GRL-TE ranked second (2.67) with no significant difference from LP ($p > 0.05$). This demonstrates that GRL-TE achieves comparable performance to the optimal LP solution while offering substantially better scalability.

These results fully validate GRL-TE's design objectives: to provide a scalable, efficient, and high-performance TE solution for modern large-scale production networks. Particularly in ultra-large-scale scenarios where traditional methods fail, GRL-TE demonstrates decisive technical advantages.

5.5 Reacting to link failures

This experiment simulates various link failure scenarios to assess the robustness of different TE methods in dynamic topologies. In the following analysis, baseline performance refers to the median demand satisfaction rate in the no-failure scenario.

In the link failure experiment on the B4 dataset (Fig. 7), we evaluated the performance of five methods in the 1-3 link failure scenarios, conducting 10 independent trials for each configuration. The box plot illustrates the distribution of demand satisfaction rates, where the box represents the interquartile range (IQR), the red line indicates the median, the square represents the mean, and the diamond marks outliers.

Results show that the LP method achieves the best performance across all failure scenarios, with the median demand satisfaction rate decreasing from 99.83% without failures to 93.09% with 3 failures (a decrease of 6.74%). The IQR ranging from 1.94% to 3.62% and absence of outliers demonstrate exceptional robustness. NCFlow performs second best, with its demand satisfaction rate decreasing from 98.90% to 88.59% (a decrease of 10.31%), showing good stability. In contrast, Teal exhibits a median of 90.89% without failures, which decreases to 73.82% in the 3-failure scenario, accompanied by significant stability issues and multiple outliers (minimum value: 16.73%), indicating potential severe degradation under specific conditions. Figret achieves near-optimal performance (99.19%) without failures but shows high sensitivity to failures, with the demand satisfaction rate decreasing to 76.18% in the 3-failure scenario (a decrease of 23.01%). GRL-TE demonstrates intermediate performance and stability, decreasing from 90.80% to 77.25%.

In the link failure experiment on the GÉANT dataset (Fig. 8), baseline performance across methods is relatively similar, with LP and NCFlow achieving medians of 88.40% and 88.99%, respectively, while other methods range between 84.25% and 86.57%. This reflects the inherent challenges of the GÉANT network topology. Regarding performance stability, LP and NCFlow again demonstrate

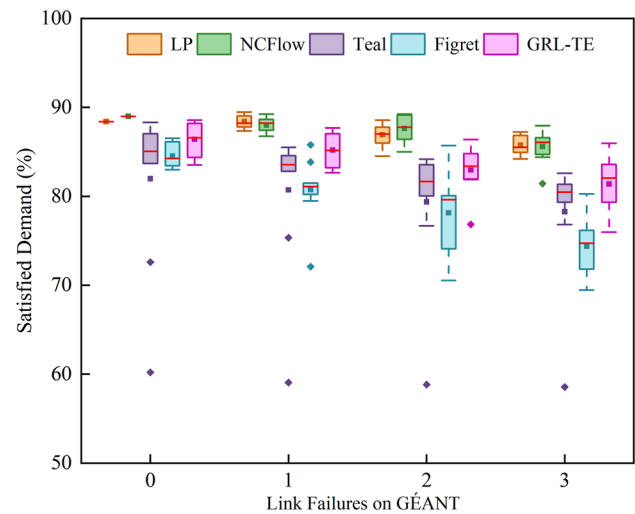


Fig. 8 Link failures on GÉANT

superior robustness, with performance decreases of 2.9% and 2.92% respectively in the 3-failure scenario, maintaining medians at 85.50% and 86.07%, with IQR consistently below 2.7%. Notably, Teal shows improved performance on GÉANT compared to B4. While outliers persist (minimum value: 58.56%), overall stability improves, with a performance decrease of 4.61% in the 3-failure scenario. Conversely, Figret exhibits poor performance on GÉANT, with low baseline performance (84.25%) and high failure sensitivity, decreasing to 74.72% (a decrease of 9.53%) in the 3-failure scenario, accompanied by an expanded interquartile range (IQR = 4.4%). GRL-TE maintains intermediate performance, decreasing from 86.57% to 82.05%, with both performance and stability between the extremes.

In the UsCarrier dataset experiment (Fig. 9), an interesting phenomenon emerges: NCFlow's baseline performance

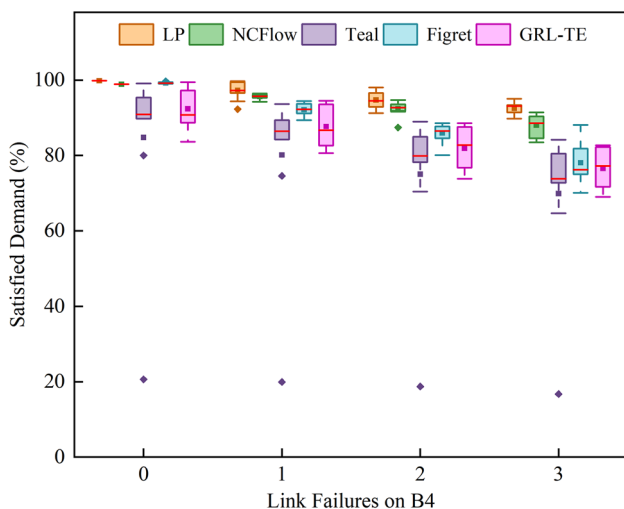


Fig. 7 Link failures on B4

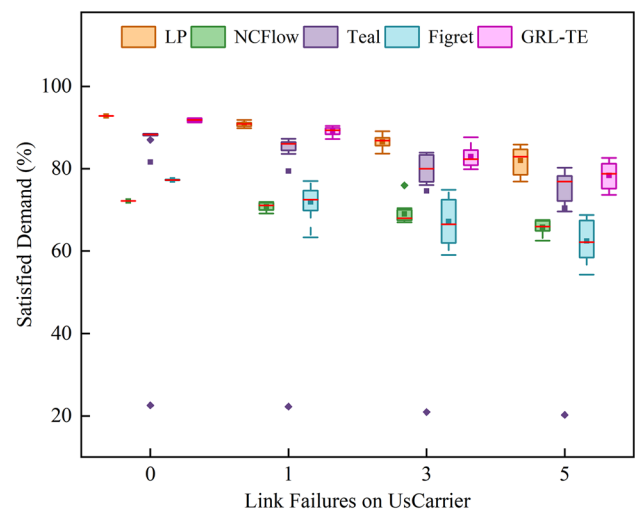


Fig. 9 Link failures on UsCarrier

(72.15%) is significantly lower than other methods, yet it demonstrates outstanding stability, decreasing to 65.95% (a decrease of 6.2%) with 5 failures, with IQR remaining below 2.56% throughout. LP continues to show the best overall performance, decreasing from 92.83% to 82.96% (a decrease of 9.87%), though IQR increases to 6.17% in the 5-failure scenario, indicating some fluctuation in extreme failure conditions. GRL-TE performs well on this dataset with baseline performance of 91.88%. However, its failure sensitivity increases, decreasing to 78.75% (a decrease of 13.13%) with 5 failures. Figret shows lower baseline performance (77.27%) and the most significant performance degradation, decreasing to 62.17% (a decrease of 15.1%) with 5 failures, and the IQR expands significantly (IQR = 8.96%). Teal continues to exhibit instability, with extreme outliers reaching 56.64% below the median in the 5-failure scenario, although median performance remains acceptable.

In the link failure experiment on the Cogentco dataset (Fig. 10), the most significant finding is the relatively small performance decreases across all methods. Even with 5 link failures, performance decreases ranges from 1.81% to 5.67%, indicating strong inherent redundancy in this network topology. LP maintains the best performance with a narrower advantage, decreasing from 89.65% to 87.84% (a decrease of 1.81%), with IQR remaining below 0.63% across all failure scenarios, demonstrating excellent stability. Teal shows significant improvement on this dataset; while outliers persist (one trial showing 68.89% performance), their frequency and impact are substantially reduced, with median performance stable between 86.86% and 83.27% (a decrease of 3.59%). NCFlow (a decrease of 3.77%) and GRL-TE (a decrease of 3.35%) exhibit similar performance patterns with good stability. Figret, despite having the lowest baseline performance (81.08%), shows marked improvement in failure sensitivity

compared to other datasets, maintaining 75.41% (a decrease of 5.67%) with 5 failures.

In the link failure experiment on the Kdl dataset (Fig. 11), we evaluated up to 10 link failures. LP achieves baseline performance of 95.40% without failures. NCFlow's baseline performance is notably low at 65.21%, significantly below other methods, yet it decreases to only 61.43% (a decrease of 3.78%) with 10 failures, demonstrating unexpected stability. In contrast, GRL-TE starting with the high performance (90.69%), decreases to 80.87% (a decrease of 9.82%) with 10 failures, exhibiting a reasonable degradation pattern. Teal shows its characteristic dual behavior, with median decreasing from 87.21% to 77.67% (a decrease of 9.54%), while retaining outliers as low as 12.1% performance, indicating persistent algorithmic instability under extreme conditions. Figret exhibits the poorest performance, with the lowest baseline performance (68.89%) and significant performance variability after 5 failures (IQR increases to 5.06%), decreasing to 61.43% with 10 failures.

In the ASN dataset link failure experiment (Fig. 12), we evaluated up to 200 link failures. NCFlow demonstrates an unexpected characteristic: despite very low baseline performance (44.78%), it decreases to only 40.10% (a decrease of 4.68%) with 200 failures, showing relative stability under extreme conditions. This low-performance, high-stability pattern likely reflects NCFlow's conservative routing strategy. In contrast, GRL-TE achieves the best performance, maintaining high baseline performance of 87.59% and 85.10% with 200 failures (a decrease of 2.49%), demonstrating the advantage of deep learning methods in handling large-scale, complex scenarios. Teal performs with surprising stability on this dataset, with median decreasing from 86.37% to 82.80% (a decrease of 3.57%), though outliers persist with performance as low as 19.84% in some trials.

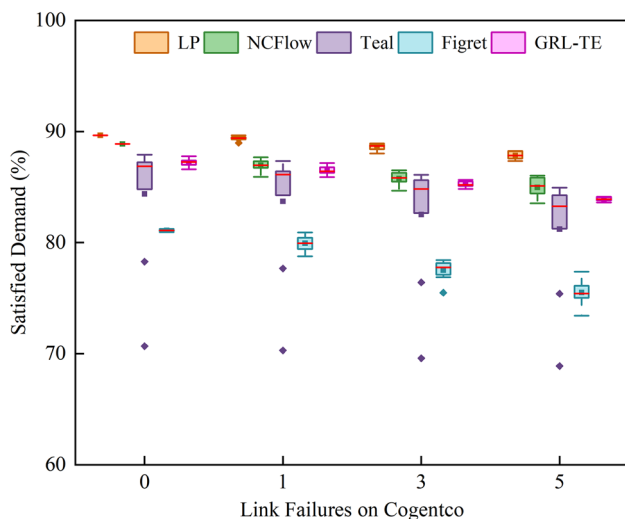


Fig. 10 Link failures on Cogentco

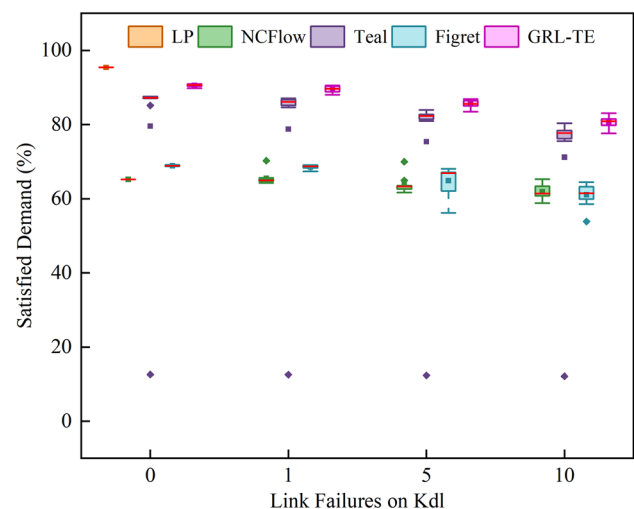


Fig. 11 Link failures on Kdl

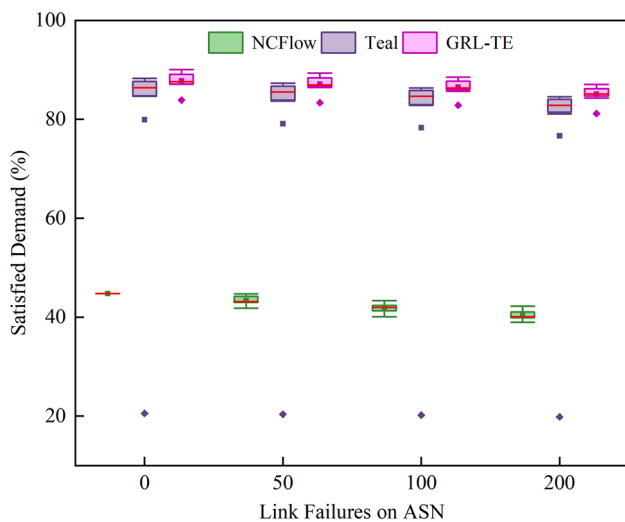


Fig. 12 Link failures on ASN

All methods maintain low IQR values ($< 2.91\%$), indicating that algorithmic behavior tends to stabilize under extreme conditions.

5.6 Ablation study of GRL-TE

To understand the contribution of each component in GRL-TE, we conducted ablation studies on six network topologies of different scales. The results of the ablation study as shown in Figs. 13, 14, and 15.

Impact of MLP module The performance of all networks showed varying degrees of degradation, with demand satisfaction rates dropping by 0.29% to 3.19%. Notably, the

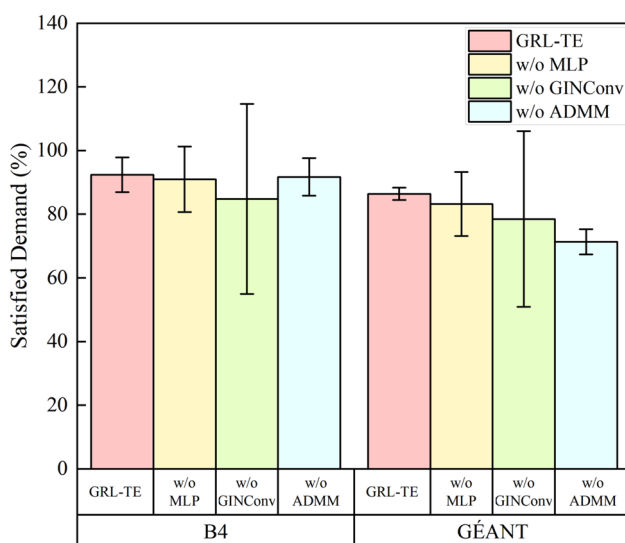


Fig. 13 Ablation study on small-scale (B4) and high-traffic (GÉANT) networks

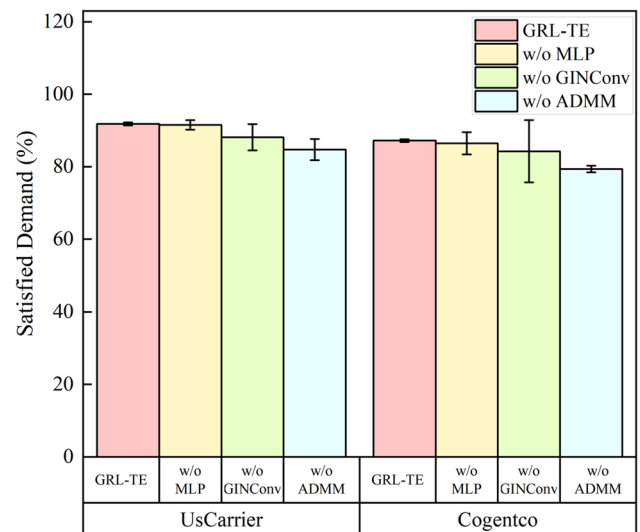


Fig. 14 Ablation study on medium-scale networks (UsCarrier and Cogentco)

standard deviation increased significantly, reaching 10.31% on B4 and 10.05% on GÉANT. This indicates that the MLP module plays a crucial role in coordinating traffic distribution among candidate paths for each demand. Without this module, the model's ability to balance traffic allocation decreases, particularly in handling collaborative relationships among multiple candidate paths for the same traffic demand, resulting in instability in traffic allocation.

Impact of GINConv module Removing the GINConv module led to a significant drop in performance across all networks, with the performance degradation ranging from 2.94% on Cogentco to 7.94% on GÉANT. The standard devi-

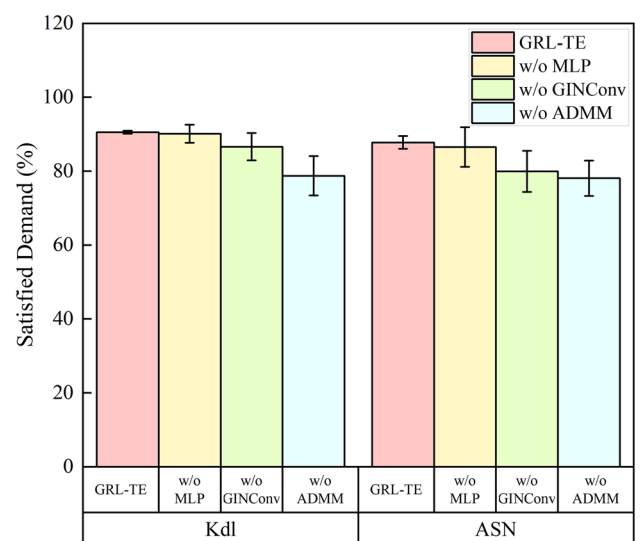


Fig. 15 Ablation study on large-scale networks (Kdl and ASN)

ation also surged dramatically, reaching 29.83% on B4 and 27.61% on GÉANT, showing catastrophic instability. The GINConv module functions by enabling bidirectional information propagation between edge nodes and path nodes in the bipartite graph structure, helping the model aggregate link capacity constraints and path demand information. Without this component, the model struggles to effectively understand network topology and resource competition, thus failing to make effective TE decisions.

Impact of ADMM The impact of the ADMM layer depends on both network scale and traffic demand complexity. In smaller networks, such as B4, removing the ADMM layer has a minimal effect, with a performance drop of only 0.68%, suggesting that the neural network can adequately handle constraints in simple topologies. However, in large-scale networks, the removal of the ADMM layer results in severe performance degradation: 7.09% on UsCarrier, 7.83% on Cogentco, 11.79% on Kdl, and 9.69% on ASN. Notably, GÉANT shows the most severe degradation at 15.09%, despite having fewer nodes than other large networks. This is because GÉANT has exceptionally high traffic demand (average demand of 43.94 billion), which creates complex optimization challenges that require ADMM's constraint handling capabilities. This pattern reveals that as network complexity increases—whether through topology scale or traffic volume—the neural network alone cannot effectively guarantee constraint satisfaction. Therefore, the ADMM layer becomes crucial for handling complex capacity and demand constraints.

Scale-dependent component importance Through the ablation study across different network scales, we identified an interesting pattern: in networks with simple constraint structures (such as B4, with low traffic volume), the GINConv module is the most critical for performance because it helps accurately capture network topology information. In networks with complex constraint challenges—either due to large topology scale (such as Kdl and ASN) or high traffic volume (such as GÉANT)—the ADMM layer becomes equally important, if not more so than GINConv. This is because, as constraint complexity grows, whether from network scale or traffic volume, the neural network alone cannot solve these complex optimization problems. The MLP module maintains consistent importance across all scales, providing necessary path coordination to ensure traffic is evenly distributed across multiple candidate paths.

The complete GRL-TE model, through the synergy of its components, achieves both high performance and exceptional stability (with a standard deviation of 0.4% to 5.45%). TopoFlowNet (GINConv + MLP) offers structured feature learning, enabling the capture of network topology and traffic patterns. The DRL module leverages these features to

generate initial traffic allocations, while ADMM refines these allocations to reduce constraint violations and ultimately provides a feasible solution.

These ablation results validate our architectural design philosophy: TopoFlowNet's graph-based feature learning provides the foundation for understanding network topology and traffic patterns, while ADMM ensures feasibility in complex scenarios. Each component plays a critical role under different complexity conditions, and their synergy creates a robust solution that maintains both high performance and stability across diverse network environments.

5.7 Training and test time comparison

We comprehensively evaluated the training and test times of each method to assess their feasibility for practical deployment. The training time reflects the one-time cost of model preparation, while the test time determines real-time performance when handling new traffic matrices. LP and NCFlow do not require training as they are optimization-based methods that directly solve each TM instance. In contrast, learning-based methods require an initial training phase to learn traffic allocation strategies before deployment.

We measured the total allocation test time for each scheme on each TM, carefully excluding initialization and other one-time overheads to ensure that the results accurately reflect computational performance. Table 7 outlines the calculation method for the total allocation time of each scheme. For instance, the test time for GRL-TE includes the total run time (with GPU), while NCFlow accounts for the Gurobi run time (Gurobi Optimization 2023) and the additional time required for merging subproblems.

Table 8 compares the training and test time of different methods across six network topologies.

The training and test time analysis results are consistent with the performance trends observed in Section 5.4, where different methods show a clear dependence on network scale in terms of efficiency.

In small-scale networks, LP demonstrates outstanding efficiency, with sub-millisecond computation times (0.0023 s for B4 and 0.012 s for GÉANT). This efficiency, combined with its excellent demand satisfaction rate (99.83% for

Table 7 Comparison of test time across different schemes

Method	Test Time
LP	Gurobi run time
NCFlow	Gurobi run time + time to coalesce subproblems
Teal	Total run time (with GPU)
Figret	Total run time (with GPU)
GRL-TE	Total run time (with GPU)

Table 8 Training time and test time on six datasets

Dataset	Model	Training Time (s)	Test Time (s)
B4	LP	–	0.0023
	NCFlow	–	0.0240
	Teal	3.8889	0.0098
	Figret	1.7310	0.0368
	GRL-TE	1.7691	0.0059
GÉANT	LP	–	0.0120
	NCFlow	–	0.0413
	Teal	1.5941	0.0543
	Figret	3.6807	0.2325
	GRL-TE	1.6503	0.1510
UsCarrier	LP	–	1.2170
	NCFlow	–	0.1990
	Teal	5.3369	0.0102
	Figret	33.9193	2.6211
	GRL-TE	5.0878	0.0718
Cogentco	LP	–	2.28
	NCFlow	–	0.1706
	Teal	7.4749	0.0475
	Figret	143.4074	34.7269
	GRL-TE	5.9887	0.0889
Kdl	LP	–	2500.6002
	NCFlow	–	0.8277
	Teal	99.7	0.3335
	Figret	829.92	75.9564
	GRL-TE	70.4	1.6817
ASN	LP	–	10499.0671
	NCFlow	–	42.3062
	Teal	263.8	0.3097
	Figret	–	–
	GRL-TE	218	1.4811

B4), confirms that traditional optimization methods are still the best choice for small-scale deployments. Learning-based methods, while showing reasonable computation times, cannot match LP's performance and are less competitive in these scenarios.

As the network size increases, the computational landscape undergoes a dramatic change. On large-scale networks, LP's computation time grows exponentially, reaching 2500.6002 s on Kdl and 10499.0671 s on ASN, making it unsuitable for real-time TE. This explosive growth in computational complexity validates the necessity of learning-based methods for large-scale networks. Among learning-based methods, GRL-TE demonstrates excellent scalability, with test times of 1.6817 s on Kdl and 1.4811 s on ASN, achieving a 3-4 order of magnitude speedup over LP while maintain-

ing competitive performance (demand satisfaction rates of 90.54% and 87.77%, respectively). Notably, while NCFlow performs well on medium-scale networks, it requires 42.3062 s on ASN, significantly higher than GRL-TE.

Training time analysis further supports the practical advantages of GRL-TE. While both GRL-TE and Teal adopt a fixed training strategy with 3 episodes, significant differences in training efficiency exist. In small-scale networks, GRL-TE and Figret have similar training times (1.7691 s and 1.7310 s on B4, respectively), both outperforming Teal (3.8889 s). As network size increases, GRL-TE's advantage becomes more apparent: on Cogentco, GRL-TE requires only 5.9887 s, while Teal requires 7.4749 s; on the largest networks, GRL-TE trains in 70.4 s on Kdl and 218 s on ASN, while Teal requires 99.7s and 263.8 s, and Figret requires 829.92 s on Kdl and is unable to train on ASN.

For production environments requiring real-time TE, especially in large-scale networks with dynamic traffic patterns, GRL-TE offers a practical solution. It balances performance, computational efficiency, and constraint satisfaction: (1) It provides a speedup by several orders of magnitude at the cost of some performance loss in large-scale networks; (2) Its reasonable training time allows models to be periodically updated to adapt to network changes. In particular, for large-scale networks requiring frequent recalculation of traffic allocations, GRL-TE's ability to generate feasible solutions within seconds makes it the only practical choice.

6 Conclusion

This article presents GRL-TE, a graph-based RL framework for TE in large-scale WANs. By combining TopoFlowNet for feature extraction, a one-step A2C mechanism for traffic allocation, and ADMM for constraint refinement, GRL-TE addresses the scalability challenges faced by traditional optimization-based TE methods.

The proposed TopoFlowNet architecture models WANs as bipartite graphs, enabling efficient information propagation between paths and links through GINConv layers, while MLP modules coordinate traffic distribution among candidate paths. This design allows the framework to capture both network topology constraints and traffic demand patterns. The integration with a one-step A2C mechanism simplifies the training process by focusing on immediate rewards, which aligns well with the nature of TE decisions. ADMM further refines the DRL-generated allocations to improve constraint satisfaction.

Experimental evaluation on six real-world topologies demonstrates that GRL-TE achieves competitive performance compared to existing methods. On average, GRL-TE attains 89.36% demand satisfaction across all tested networks, outperforming learning-based baselines while pro-

viding significant computational speedups over LP solvers in large-scale scenarios. The framework shows reasonable robustness to link failures and maintains stable performance across different network scales.

Looking ahead, future research can extend GRL-TE in several directions. Multi-objective optimization incorporating factors such as latency, jitter, and energy consumption could address a broader range of demands in WAN traffic scheduling through carefully designed composite reward functions.

Additionally, enhancing GRL-TE's adaptability and robustness presents important research opportunities. Inspired by recent advances in uncertainty modeling, such as the variance-constrained approaches in device-free localization (Zhang et al. 2023), incorporating uncertainty-aware mechanisms could help GRL-TE better handle incomplete or noisy network state information in dynamic WAN environments. Furthermore, drawing insights from online learning techniques successfully applied in Wi-Fi-based positioning systems (Zhang et al. 2025), future versions of GRL-TE could dynamically adapt to evolving network topologies and traffic patterns without requiring extensive retraining.

By addressing these directions and leveraging cross-domain insights, GRL-TE can evolve into a more robust and adaptive solution for next-generation network infrastructure, capable of handling the increasing complexity and dynamism of modern WANs.

Acknowledgements This work was supported by the National Key Research and Development Program of China (No.2022YFB2901400, 2023YFB2904100).

Author Contributions Conceptualization: Chaowei Tang; Methodology: Jingwen Lu, Chaowei Tang; Formal analysis and investigation: Jingwen Lu, Wenyu Ma; Writing – original draft preparation: Jingwen Lu; Writing – review and editing: Chaowei Tang, Wenjuan Xing; Funding acquisition: Chaowei Tang, Wenjuan Xing; Resources: Wenjuan Xing; Supervision: Chaowei Tang. All authors read and approved the final manuscript.

Availability of Data and Material Publicly available datasets were used in this study, including WAN topologies such as B4, UsCarrier, Kdl, ASN, GÉANT, and Cogentco. These datasets are cited in the manuscript and accessible via the referenced sources. Additional supporting data are available from the corresponding author upon reasonable request.

Declarations

Competing Interests The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indi-

cate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

References

- Abuzaid F, Kandula S, Arzani B, Menache I, Zaharia M, Bailis P (2021) Contracting wide-area network topologies to solve flow problems quickly. In: 18th USENIX Symposium on Networked Systems Design and Implementation (NSDI 21), pp 175–200
- Applegate D, Cohen E (2003) Making intra-domain routing robust to changing and uncertain traffic demands: understanding fundamental tradeoffs. In: Proceedings of the 2003 conference on applications, technologies, architectures, and protocols for computer communications, pp 313–324
- Awduche D, Chiu A, Elwalid A, Widjaja I, Xiao X (2002) Overview and principles of internet traffic engineering. Technical report
- Bernárdez G, Suárez-Varela J, López A, Wu B, Xiao S, Cheng X, Barlet-Ros P, Cabellos-Aparicio A (2021) Is machine learning ready for traffic engineering optimization? In: 2021 IEEE 29th International Conference on Network Protocols (ICNP), pp 1–11. IEEE
- Bogle J, Bhatia N, Ghobadi M, Menache I, Bjørner N, Valadarsky A, Schapira M (2019) Teavar: striking the right utilization-availability balance in wan traffic engineering. In: Proceedings of the ACM special interest group on data communication, pp 29–43
- CAIDA (2022) The CAIDA AS relationships dataset. <http://www.caida.org/data/active/as-relationships/>
- Diederik PK (2014) Adam: a method for stochastic optimization. (No Title)
- Ding M, Guo Y, Huang Z, Lin B, Luo H (2024) Grom: a generalized routing optimization method with graph neural network and deep reinforcement learning. *J Netw Comput Appl* 229:103927
- Fortz B, Rexford J, Thorup M (2002) Traffic engineering with traditional ip routing protocols. *IEEE Commun Mag* 40(10):118–124
- Fortz B, Thorup M (2000) Internet traffic engineering by optimizing ospf weights. In: Proceedings IEEE INFOCOM 2000. Conference on computer communications. Nineteenth annual joint conference of the IEEE computer and communications societies (Cat. No. 00CH37064), vol 2, pp 519–528. IEEE
- Geng N, Xu M, Yang Y, Liu C, Yang J, Li Q, Zhang S (2021) Distributed and adaptive traffic engineering with deep reinforcement learning. In: 2021 IEEE/ACM 29th International Symposium on Quality of Service (IWQOS), pp 1–10. IEEE
- Gui F, Wang S, Li D, Chen L, Gao K, Min C, Wang Y (2024) Redte: mitigating subsecond traffic bursts with real-time and distributed traffic engineering. In: Proceedings of the ACM SIGCOMM 2024 conference, pp 71–85
- Guo Y, Wang W, Zhang H, Guo W, Wang Z, Tian Y, Yin X, Wu J (2021) Traffic engineering in hybrid software defined network via reinforcement learning. *J Netw Comput Appl* 189:103116
- Guo Y, Ding M, Zhou W, Lin B, Chen C, Luo H (2024) Mate: a multi-agent reinforcement learning approach for traffic engineering in hybrid software defined networks. *J Netw Comput Appl* 231:103981
- Guo Y, Zhou W, Hu C, Lin B, Deng Y, Min G (2025) Dite: achieving distributed intelligent traffic engineering with sparse network-wide guidance. *IEEE Trans Netw Sci Eng*
- Gurobi Optimization L (2023) Gurobi optimizer reference manual. <https://www.gurobi.com>

- Hong C-Y, Kandula S, Mahajan R, Zhang M, Gill V, Nanduri M, Wattenhofer R (2013) Achieving high utilization with software-driven wan. In: Proceedings of the ACM SIGCOMM 2013 conference on SIGCOMM, pp 15–26
- Hong C-Y, Mandal S, Al-Fares M, Zhu M, Alimi R, Bhagat C, Jain S, Kaimal J, Liang S, Mendelev K et al (2018) B4 and after: managing hierarchy, partitioning, and asymmetry for availability and scale in google's software-defined wan. In: Proceedings of the 2018 conference of the ACM special interest group on data communication, pp 74–87
- Jain S, Kumar A, Mandal S, Ong J, Poutievski L, Singh A, Venkata S, Wanderer J, Zhou J, Zhu M et al (2013) B4: experience with a globally-deployed software defined wan. *ACM SIGCOMM Comput Commun Rev* 43(4):3–14
- Jamaludin SZM, Romli NA, Kasihmuddin MSM, Baharum A, Mansor MA, Marsani MF (2022) Novel logic mining incorporating log linear approach. *J King Saud Univ-Comput Inf Sci* 34(10):9011–9027
- Jin X, Li Y, Wei D, Li S, Gao J, Xu L, Li G, Xu W, Rexford J (2016) Optimizing bulk transfers with software-defined optical wan. In: Proceedings of the 2016 ACM SIGCOMM conference, pp 87–100
- Knight S, Nguyen HX, Falkner N, Bowden R, Roughan M (2011) The internet topology zoo. *IEEE J Sel Areas Commun* 29(9):1765–1775
- Krishnaswamy U, Singh R, Bjørner N, Raj H (2022) Decentralized cloud wide-area network traffic engineering with {BLASTSHIELD}. In: 19th USENIX Symposium on Networked Systems Design and Implementation (NSDI 22), pp 325–338
- Laoutaris N, Sirivianos M, Yang X, Rodriguez P (2011) Inter-datacenter bulk transfers with Netstitcher. In: Proceedings of the ACM SIGCOMM 2011 conference, pp 74–85
- Liu HH, Kandula S, Mahajan R, Zhang M, Gelernter D (2014) Traffic engineering with forward fault correction. In: Proceedings of the 2014 ACM conference on SIGCOMM, pp 527–538
- Liu X, Zhao S, Cui Y, Wang X (2024) Figret: fine-grained robustness-enhanced traffic engineering. In: Proceedings of the ACM SIGCOMM 2024 conference, pp 117–135
- Lu J, Tang C, Chen Z, Guo J, Zou A, Yang W, Tang C (2025) Intraflow temporal correlation-based network traffic prediction. *Comput Netw* 256:110913
- Ma Y, Guo Y, Yang R, Luo H (2025) Frrl: a reinforcement learning approach for link failure recovery in a hybrid sdn. *J Netw Comput Appl* 234:104054
- Marouani S, Singh K, Jeudy B, Bradai A, Habrard A (2024) Advanced traffic engineering in wan using graph attention networks. In: 2024 20th International conference on Wireless and Mobile Computing, Networking and Communications (WiMob), pp 657–662. IEEE
- McKeown N, Anderson T, Balakrishnan H, Parulkar G, Peterson L, Rexford J, Shenker S, Turner J (2008) Openflow: enabling innovation in campus networks. *ACM SIGCOMM Comput Commun Rev* 38(2):69–74
- Mehraban S, Yadav RK (2024) Traffic engineering and quality of service in hybrid software defined networks. *China Commun* 21(2):96–121
- Nance-Hall M, Barford P, Foerster K-T, Durairajan R (2023) Improving scalability in traffic engineering via optical topology programming. *IEEE Trans Netw Serv Manag*
- Narayanan D, Kazhamiaka F, Abuzaid F, Kraft P, Agrawal A, Kandula S, Boyd S, Zaharia M (2021) Solving large-scale granular resource allocation problems efficiently with pop. In: Proceedings of the ACM SIGOPS 28th symposium on operating systems principles, pp 521–537
- Roughan M, Greenberg A, Kalmanek C, Rumsewicz M, Yates J, Zhang Y (2002) Experience in measuring backbone traffic variability: models, metrics, measurements and meaning. In: Proceedings of the 2nd ACM SIGCOMM workshop on internet measurement, pp 91–92
- Rusdi N, Kasihmuddin MSM, Romli NA, Manoharam G, Mansor MA (2023) Multi-unit discrete hopfield neural network for higher order supervised learning through logic mining: optimal performance design and attribute selection. *J King Saud Univ-Comput Inf Sci* 35(5):101554
- Rusek K, Suárez-Varela J, Mestres A, Barlet-Ros P, Cabellos-Aparicio A (2019) Unveiling the potential of graph neural networks for network modeling and optimization in sdn. In: Proceedings of the 2019 ACM symposium on SDN research, pp 140–151
- Saxena MC, Sabharwal M, Bajaj P (2023) Comparative study and proposal to use enhanced bellman-ford algorithm for faster path computation on sdwan controller. *J Theor Appl Inf Technol* 101(16)
- Schlinker B, Kim H, Cui T, Katz-Bassett E, Madhyastha HV, Cunha I, Quinn J, Hasan S, Lapukhov P, Zeng H (2017) Engineering egress with edge fabric: steering oceans of content to the world. In: Proceedings of the conference of the ACM special interest group on data communication, pp 418–431
- Schrittwieser J, Antonoglou I, Hubert T, Simonyan K, Sifre L, Schmitt S, Guez A, Lockhart E, Hassabis D, Graepel T et al (2020) Mastering Atari, go, chess and shogi by planning with a learned model. *Nature* 588(7839):604–609
- Swaminathan A, Chaba M, Sharma DK, Ghosh U (2021) Graphnet: graph neural networks for routing optimization in software defined networks. *Comput Commun* 178:169–182
- Tian Y, Wang Z, Yin X, Shi X, Guo Y, Geng H, Yang J (2020) Traffic engineering in partially deployed segment routing over ipv6 network with deep reinforcement learning. *IEEE/ACM Trans Netw* 28(4):1573–1586
- Tinhofer G, Tinhofer G (1976) Graphen und matrisen. *Methoden der angewandten Graphentheorie*, pp 73–93
- Uhlig S, Quoitin B, Lepropre J, Balon S (2006) Providing public intradomain traffic matrices to the research community. *ACM SIGCOMM Comput Commun Rev* 36(1):83–86
- Xu Z, Yan FY, Singh R, Chiu JT, Rush AM, Yu M et al (2023) Learning-accelerated optimization of wan traffic engineering. In: Proceedings of the ACM SIGCOMM 2023 conference, pp 378–393
- Yap K-K, Motiwala M, Rahe J, Padgett S, Holliman M, Baldus G, Hines M, Kim T, Narayanan A, Jain A et al (2017) Taking the edge off with espresso: scale, reliability and programmability for global internet peering. In: Proceedings of the conference of the ACM special interest group on data communication, pp 432–445
- Zhang J, Li Y, Li Q, Xiao W (2023) Variance-constrained local-global modeling for device-free localization under uncertainties. *IEEE Trans Ind Inform* 20(4):5229–5240
- Zhang M, Liu B, Zhang B (2009) Multi-commodity flow traffic engineering with hybrid mpls/ospf routing. In: GLOBECOM 2009-2009 IEEE global telecommunications conference, pp 1–6. IEEE
- Zhang J, Xue J, Li Y, Cotton SL (2025) Leveraging online learning for domain-adaptation in wi-fi-based device-free localization. *IEEE Trans Mob Comput*